

Índice general

1	Introducción	1
2	Estado del Arte	2
2.1	Sistemas RAG (Retrieval-Augmented Generation)	2
2.2	Web Scraping de Wikis y Contenido Colaborativo	2
2.3	NLP en Dominios Específicos	3
2.4	Traducción Automática y Multilingüismo	4
2.5	Algoritmos de Búsqueda: BM25	4
2.6	Brechas Identificadas y Justificación del Proyecto	5
3	Corpus y Recopilación de Datos	6
3.1	Fuente de Datos: Liquipedia	6
3.2	Proceso de Web Scraping	6
3.2.1	Navegación Automatizada con Selenium	7
3.2.2	Extracción de Información con BeautifulSoup	7
3.2.3	Selección de Equipos	8
3.3	Preprocesamiento y Limpieza	8
3.3.1	Normalización de HTML	8
3.3.2	Traducción Automática	9
3.3.3	Validación de Calidad	9
3.4	Características del Corpus Final	10
3.4.1	Estructura de Datos	10
3.5	Consideraciones Éticas y Legales	11
3.6	Limitaciones del Corpus	11
4	Análisis Exploratorio de Datos (EDA)	12
4.1	Estadísticas Básicas del Corpus	12

4.2	Análisis de Calidad de Datos	12
4.3	Distribución de Longitud de Documentos	13
4.3.1	Interpretación	14
4.4	Distribución de Palabras por Documento	14
4.4.1	Observaciones Principales	14
4.5	Análisis de Frecuencias de Términos	15
4.5.1	Términos Dominantes	15
4.5.2	Implicaciones	17
4.6	Densidad de Oraciones por Documento	17
4.6.1	Análisis de Densidad	17
4.7	Análisis de Calidad Textual	18
4.7.1	Detección de Repeticiones	18
4.7.2	Análisis de Encoding	19
4.7.3	Proporción Español-Inglés	19
4.8	Identificación de Patrones Relevantes	19
4.8.1	Estructuras Recurrentes	19
4.8.2	Información Temporal	20
4.8.3	Entidades Nombradas	20
4.9	Evaluación Preliminar del Sistema de Recuperación	20
4.9.1	Métricas Seleccionadas	20
4.9.2	Conjunto de Evaluación	21
4.9.3	Resultados de la Evaluación	22
4.9.4	Relación con los Objetivos del Proyecto	23
4.10	Conclusiones del EDA	23
5	Identificación de Sesgos	25
5.1	Sesgo de Popularidad y Cobertura	25
5.2	Sesgo Temporal	25
5.3	Sesgo de Fuente Única	26
5.4	Sesgo Lingüístico por Traducción Automática	26
5.5	Sesgo de Selección y Notabilidad	27
5.6	Sesgo Geográfico	27
5.7	Estrategias de Mitigación	27
5.7.1	Mitigación del Sesgo de Popularidad	28
5.7.2	Mitigación del Sesgo Temporal	28

5.7.3	Mitigación del Sesgo Lingüístico	28
5.7.4	Transparencia y Comunicación de Limitaciones	29
6	Metodología	30
6.1	Descripción General del Sistema	30
6.2	Preprocesamiento	30
6.2.1	Web Scraping con Selenium	31
6.2.2	Limpieza de Texto	32
6.2.3	Traducción Automática	33
6.3	Arquitectura del Sistema RAG	33
6.3.1	Almacén Vectorial y Tokenización	33
6.3.2	Expansión de Consultas	34
6.3.3	Algoritmo BM25 para Recuperación	35
6.3.4	Pipeline de Ingesta	35
6.3.5	Generador Conversacional	36
6.3.6	Consideraciones de Diseño	37
6.4	Métricas de Evaluación	38
6.4.1	BERTScore para Evaluación Semántica	38
6.4.2	MRR para Evaluación de Recuperación	39
6.4.3	Conjunto de Evaluación	39
6.4.4	Protocolo de Evaluación	40
6.4.5	Justificación de Selección de Métricas	40
7	Resultados	42
7.1	Corpus Final y Características	42
7.1.1	Estadísticas Descriptivas del Corpus	42
7.1.2	Preparación de Documentos para Indexación	43
7.2	Configuración del Sistema de Recuperación	43
7.2.1	Parámetros de Recuperación	44
7.3	Evaluación del Sistema de Recuperación	44
7.3.1	Conjunto de Evaluación	44
7.3.2	Métricas de Evaluación	45
7.3.3	Resultados Globales	45
7.3.4	Resultados por Tipo de Consulta	46

7.4	Análisis de Rendimiento	47
7.4.1	Fortalezas del Sistema	47
7.4.2	Debilidades Identificadas	47
7.4.3	Casos de Éxito	48
7.4.4	Casos Problemáticos	48
7.4.5	Comparación con Sistemas Similares	49
8	Discusión	50
8.1	Interpretación de Resultados	50
8.1.1	Rendimiento en Consultas Estructuradas	50
8.1.2	Limitaciones en Consultas No Estructuradas	50
8.1.3	Eficiencia Computacional	51
8.2	Fortalezas y Limitaciones	51
8.2.1	Fortalezas del Sistema	51
8.2.2	Limitaciones del Sistema	52
8.3	Trabajo Futuro	54
8.3.1	Transición a Embeddings Semánticos	54
8.3.2	Integración de Modelos de Lenguaje Más Grandes	55
8.3.3	Ampliación del Corpus con Fuentes Adicionales	55
8.3.4	Incorporación de Razonamiento Temporal	55
8.3.5	Evaluación con Usuarios Reales	56
8.4	Aplicaciones Prácticas	56
8.5	Reflexiones sobre el Proceso de Desarrollo	56
8.6	Conclusiones	57
	Bibliografía	59

1. INTRODUCCIÓN

El crecimiento exponencial de los deportes electrónicos (esports) en la última década ha generado un volumen masivo de información distribuida en múltiples plataformas digitales. En particular, el ecosistema competitivo de *Counter-Strike*, uno de los juegos más longevos y populares en la escena profesional, cuenta con cientos de equipos, miles de jugadores y un constante flujo de fichajes, resultados de torneos y cambios en las plantillas. Esta información, aunque valiosa para aficionados, analistas y profesionales del sector, se encuentra fragmentada en diversas fuentes como sitios web especializados, redes sociales y wikis colaborativas.

Liquipedia se ha consolidado como la enciclopedia colaborativa de referencia en esports, mantenida por la comunidad y estructurada de forma similar a Wikipedia. Sin embargo, el acceso a esta información requiere navegación manual entre múltiples páginas, lo que dificulta la obtención rápida de respuestas a consultas específicas. En este contexto, los sistemas de pregunta-respuesta conversacionales basados en Procesamiento del Lenguaje Natural (PLN) representan una oportunidad para facilitar el acceso a información estructurada mediante interfaces intuitivas en lenguaje natural.

El presente trabajo aborda el desarrollo de un sistema completo de minería de texto y PLN que integra técnicas de web scraping, preprocesamiento lingüístico, traducción automática y generación aumentada por recuperación (RAG, *Retrieval-Augmented Generation*). El sistema resultante permite a los usuarios consultar información sobre equipos profesionales de *Counter-Strike* en español de forma conversacional, superando las barreras idiomáticas y de navegación que presentan las fuentes originales.

Los objetivos principales de este proyecto son:

- Construir un corpus en español sobre equipos profesionales de *Counter-Strike* mediante web scraping automatizado de Liquipedia.
- Implementar un pipeline de preprocesamiento que incluya limpieza, normalización y traducción automática de textos.
- Desarrollar un sistema RAG que combine búsqueda semántica con modelos de lenguaje para generar respuestas contextualizadas.
- Analizar los sesgos presentes en el corpus y proponer estrategias de mitigación.
- Evaluar el rendimiento del sistema mediante métricas estándar de recuperación y generación.

2. ESTADO DEL ARTE

En este capítulo se revisan los trabajos y técnicas existentes que fundamentan el desarrollo de este proyecto, abordando cuatro áreas principales: sistemas de generación aumentada por recuperación (RAG), web scraping de recursos colaborativos, procesamiento de lenguaje natural en dominios específicos, y traducción automática. La combinación de estas técnicas constituye la base metodológica del chatbot propuesto.

2.1 SISTEMAS RAG (RETRIEVAL-AUGMENTED GENERATION)

Los sistemas de generación aumentada por recuperación (RAG) representan un paradigma que combina la recuperación de información con modelos de lenguaje generativos. [1] introdujeron este enfoque en su trabajo seminal, demostrando que la incorporación de un mecanismo de recuperación de documentos relevantes mejora significativamente la capacidad de los modelos de lenguaje para responder preguntas basándose en conocimiento factual externo.

El funcionamiento básico de un sistema RAG consiste en dos componentes principales: un recuperador (*retriever*) que busca documentos relevantes dado una consulta, y un generador que produce respuestas coherentes basándose en los documentos recuperados [2]. Esta arquitectura resulta especialmente útil para dominios con conocimiento dinámico y actualizable, como es el caso de los esports, donde la información sobre fichajes y equipos cambia constantemente.

[3] demostraron que el rendimiento de sistemas RAG mejora significativamente cuando se utilizan múltiples pasajes recuperados, una técnica conocida como Fusion-in-Decoder (FiD). Por su parte, [4] propusieron DPR (Dense Passage Retrieval), que utiliza representaciones densas para la recuperación de información, superando a métodos tradicionales como BM25 en ciertos escenarios.

Sin embargo, el presente proyecto utiliza BM25 como algoritmo de recuperación debido a su efectividad demostrada en corpus de tamaño moderado, su eficiencia computacional y su independencia de modelos de lenguaje pre-entrenados que requieren recursos significativos [5].

2.2 WEB SCRAPING DE WIKIS Y CONTENIDO COLABORATIVO

La extracción automatizada de información desde sitios web colaborativos como Wikipedia y Liquipedia presenta desafíos técnicos específicos. [6] analizan las consideraciones éticas y técnicas del scraping de wikis, des-

tacando la importancia de respetar los términos de uso y las buenas prácticas de acceso automatizado.

Liquipedia, como wiki especializada en esports, comparte características estructurales con Wikipedia pero presenta particularidades propias: contenido altamente dinámico, actualizaciones frecuentes realizadas por la comunidad, y ausencia de API oficial [7]. Esto obliga a utilizar técnicas de scraping basadas en análisis del DOM HTML y navegación automatizada.

[8] identifican tres desafíos principales en el web scraping moderno: contenido generado dinámicamente mediante JavaScript, sistemas anti-bot, y estructura HTML inconsistente. En el contexto de Liquipedia, el contenido se carga parcialmente mediante JavaScript, lo que requiere el uso de navegadores headless como Selenium para garantizar la captura completa de información [9].

[10] enfatizan la importancia de la normalización y limpieza de datos extraídos mediante scraping, especialmente cuando provienen de contenido generado por usuarios. Las wikis colaborativas pueden contener inconsistencias en formato, enlaces rotos y artefactos de edición que deben tratarse en el preprocesamiento.

2.3 NLP EN DOMINIOS ESPECÍFICOS

La aplicación de procesamiento de lenguaje natural a dominios específicos como los esports presenta desafíos particulares relacionados con el vocabulario especializado, jerga técnica y entidades nombradas propias del dominio [11].

[12] demostraron que los modelos de lenguaje pre-entrenados en textos generales (como BERT) obtienen mejores resultados cuando se afinan (*fine-tuning*) con corpus específicos del dominio. Sin embargo, esta aproximación requiere grandes cantidades de datos etiquetados del dominio, un requisito no siempre viable en áreas especializadas como los esports profesionales.

Los sistemas de pregunta-respuesta en dominios específicos enfrentan el desafío de la ambigüedad terminológica. [13] introdujeron SQuAD, un benchmark ampliamente utilizado para sistemas de pregunta-respuesta, aunque reconocen que el rendimiento en dominios específicos suele ser inferior debido a la falta de representatividad del corpus de entrenamiento.

En el contexto de esports, nombres de equipos, apodos de jugadores (*nicknames*) y términos técnicos del juego constituyen un vocabulario especializado que puede no estar bien representado en modelos de lenguaje generalistas [14]. Por ello, el presente proyecto enfatiza la construcción de un corpus específico y actualizado desde Liquipedia.

2.4 TRADUCCIÓN AUTOMÁTICA Y MULTILINGÜISMO

Dado que Liquipedia está principalmente en inglés y el objetivo del proyecto es un chatbot en español, la traducción automática constituye un componente crítico del pipeline de procesamiento.

Los sistemas modernos de traducción automática neural (NMT) han alcanzado niveles de calidad cercanos a la traducción humana en pares de idiomas bien representados como inglés-español [15]. [16] introdujeron la arquitectura Transformer, que revolucionó la traducción automática y constituye la base de sistemas actuales como Google Translate y DeepL.

Sin embargo, la traducción de textos especializados presenta desafíos específicos. [17] demostraron que los sistemas NMT tienden a cometer más errores en vocabulario específico de dominio, especialmente con entidades nombradas y términos técnicos. En el contexto de esports, nombres de equipos como "G2 Esports", apodos de jugadores como "NiKoz" términos técnicos del juego deben preservarse sin traducción.

[18] analizan el impacto de errores de traducción automática en sistemas posteriores de procesamiento, concluyendo que errores en entidades nombradas pueden degradar significativamente el rendimiento de sistemas de recuperación de información. Por ello, el preprocesamiento cuidadoso y la validación de traducciones son críticos en este proyecto.

2.5 ALGORITMOS DE BÚSQUEDA: BM25

BM25 (Best Matching 25) es un algoritmo de ranking probabilístico ampliamente utilizado en recuperación de información [5]. Basado en el modelo probabilístico de relevancia de Robertson y Spärck Jones, BM25 mejora TF-IDF mediante la saturación de la frecuencia de términos y la normalización por longitud de documento.

[19] analizaron diversas variantes de BM25, concluyendo que sigue siendo altamente competitivo frente a métodos de recuperación más modernos, especialmente en corpus de tamaño pequeño a mediano (menos de 100,000 documentos). Para el presente proyecto, con aproximadamente 30 documentos correspondientes a equipos de Counter-Strike, BM25 ofrece un equilibrio óptimo entre eficacia y eficiencia computacional.

[20] compararon BM25 con métodos de recuperación densos basados en transformers, encontrando que BM25 mantiene ventajas en consultas cortas y cuando el vocabulario de consulta coincide con el vocabulario del corpus, características presentes en preguntas sobre equipos y jugadores específicos.

2.6 BRECHAS IDENTIFICADAS Y JUSTIFICACIÓN DEL PROYECTO

La revisión de la literatura revela varias brechas que justifican el presente proyecto:

- **Ausencia de recursos NLP para esports en español:** La mayoría de trabajos sobre NLP aplicado a esports se centran en inglés, dejando el español desatendido [14].
- **Falta de corpus estructurados sobre fichajes:** Aunque existen bases de datos sobre estadísticas de juego, la información sobre movimientos de jugadores entre equipos está dispersa y no estructurada.
- **Sistemas RAG en dominios de alta actualización:** Los trabajos sobre RAG suelen evaluarse en corpus estáticos (Wikipedia, libros), pero no en dominios con actualizaciones diarias como los esports profesionales.
- **Traducción automática de contenido especializado:** Pocos estudios abordan la traducción de textos de esports con su vocabulario técnico específico.

Este proyecto contribuye a llenar estas brechas mediante la construcción de un sistema RAG en español basado en datos actualizados de Liquipedia, combinando técnicas de scraping, traducción automática y recuperación de información para un dominio especializado y dinámico.

3. CORPUS Y RECOPIACIÓN DE DATOS

Este capítulo describe el proceso completo de construcción del corpus del proyecto, desde la selección de la fuente de datos hasta las características finales del dataset obtenido. El corpus constituye la base de conocimiento sobre la cual opera el sistema RAG propuesto.

3.1 FUENTE DE DATOS: LIQUIPEDIA

Liquipedia (<https://liquipedia.net>) es una enciclopedia colaborativa especializada en deportes electrónicos, mantenida por la comunidad mediante un sistema wiki similar a Wikipedia. Se seleccionó Liquipedia como fuente principal de datos por las siguientes razones:

- **Exhaustividad:** Liquipedia contiene información detallada sobre equipos profesionales, incluyendo historiales de jugadores, cambios de roster, logros competitivos y biografías.
- **Actualización continua:** La comunidad mantiene la información actualizada en tiempo real, con cambios reflejados frecuentemente tras anuncios oficiales de equipos.
- **Estructura consistente:** Aunque es contenido generado por usuarios, Liquipedia mantiene plantillas estandarizadas que facilitan la extracción automatizada.
- **Verificabilidad:** Las entradas incluyen referencias a fuentes externas (Twitter, medios especializados) que validan la información.
- **Multilingüismo:** Aunque predomina el inglés, la información es consistente entre diferentes idiomas de esports.

Se eligió Counter-Strike como el juego objetivo debido a su madurez como sport (más de 20 años de historia competitiva), la abundancia de información disponible en Liquipedia, y la estabilidad de sus equipos profesionales comparado con títulos más recientes.

3.2 PROCESO DE WEB SCRAPING

Dado que Liquipedia no ofrece una API oficial, se implementó un sistema de web scraping automatizado utilizando Python con las bibliotecas Selenium y BeautifulSoup. El proceso consta de las siguientes etapas:

3.2.1 Navegación Automatizada con Selenium

Se utilizó Selenium WebDriver con Google Chrome en modo headless por las siguientes razones técnicas:

- **Contenido dinámico:** Liquipedia carga parte de su contenido mediante JavaScript, especialmente tablas de transferencias y estadísticas. Selenium permite esperar a que el JavaScript se ejecute completamente antes de extraer el HTML.
- **Scroll programático:** Para garantizar la carga de todos los elementos (lazy loading), se implementó scroll automático hacia diferentes posiciones de la página.
- **Gestión de recursos:** Se crea una nueva instancia del navegador para cada URL scrapeada, evitando fugas de memoria en procesos largos.

El código implementa las siguientes optimizaciones:

- Timeout de carga: 20 segundos máximo por página
- Scroll progresivo: mitad de página → final de página
- User-Agent personalizado: evitar bloqueos anti-bot
- Headless mode: ejecución sin interfaz gráfica

3.2.2 Extracción de Información con BeautifulSoup

Una vez obtenido el HTML completo, se utiliza BeautifulSoup para parsear y extraer información estructurada. Los elementos extraídos incluyen:

1. **Infobox del equipo:** Ubicación, fecha de fundación, entrenadores, manager.
2. **Texto principal:** Biografía del equipo, logros históricos, filosofía organizacional.
3. **Tablas de roster:** Jugadores actuales con sus roles (AWPer, rifler, IGL).
4. **Historial de transferencias:** Movimientos de jugadores con fechas y equipos origen/destino.
5. **Referencias externas:** Enlaces a anuncios oficiales en Twitter, HLTV.org y medios especializados.

Se implementó un sistema de filtrado de referencias que conserva solo enlaces a fuentes verificables, excluyen-

do enlaces internos de Liquipedia y dominios no relevantes:

Fuentes válidas incluidas:

- x.com / twitter.com (anuncios oficiales)
- hltv.org (base de datos de Counter-Strike)
- vlr.gg (estadísticas de esports)
- facebook.com, instagram.com (comunicados oficiales)
- medios especializados (dexerto, dust2.us, etc.)

3.2.3 Selección de Equipos

Se seleccionaron 30 equipos de Counter-Strike siguiendo criterios de relevancia y diversidad geográfica:

- **Equipos tier 1:** Organizaciones de élite con presencia constante en torneos principales (G2 Esports, Natus Vincere, FaZe Clan, etc.).
- **Equipos tier 2–3:** Organizaciones emergentes o regionales para aumentar diversidad del corpus.
- **Diversidad geográfica:** Equipos de Europa, América, Asia para capturar diferentes contextos organizacionales.
- **Historiales variados:** Mezcla de equipos consolidados y organizaciones recientes.

La lista completa de equipos scrapeados se puede consultar en el archivo `data/latest_ingest.json`.

3.3 PREPROCESAMIENTO Y LIMPIEZA

Los textos extraídos mediante scraping requieren un preprocesamiento exhaustivo antes de ser utilizables en el sistema RAG. El pipeline de limpieza implementado incluye:

3.3.1 Normalización de HTML

- Eliminación de etiquetas HTML residuales (`<span>`, `<div>`, etc.).
- Conversión de entidades HTML (` `, `&`, etc.) a caracteres Unicode.

- Eliminación de artefactos de navegación (“saltar”, “editar”, “ver historial”).
- Normalización de espacios en blanco: múltiples espacios ->espacio simple, múltiples saltos de línea ->máximo dos.

3.3.2 Traducción Automática

Dado que el objetivo es un chatbot en español y Liquipedia está en inglés, se implementó traducción automática mediante la biblioteca `deep-translator` con Google Translate como backend. Las consideraciones técnicas incluyen:

- **Traducción por chunks:** El texto se divide en fragmentos de máximo 1000 caracteres para evitar límites de la API y mejorar la granularidad.
- **Preservación de entidades:** Aunque no se implementó un sistema específico de protección de entidades nombradas, se validó manualmente que nombres de equipos y jugadores se preservan correctamente en la mayoría de casos.
- **Fallback:** Si la traducción de un chunk falla, se conserva el texto original en inglés en lugar de descartar información.
- **Detección de errores:** Se valida que el texto traducido tenga longitud razonable (no vacío, no excesivamente corto).

3.3.3 Validación de Calidad

Se implementaron validaciones automáticas para garantizar la calidad del corpus:

- Longitud mínima: 100 caracteres por documento
- Longitud máxima: 100,000 caracteres (detección de errores)
- Ausencia de fragmentos HTML: verificación mediante regex
- Coherencia de encoding: UTF-8 válido

Los documentos que no superan las validaciones se registran en logs pero no se descartan completamente, permitiendo revisión manual posterior.

3.4 CARACTERÍSTICAS DEL CORPUS FINAL

El corpus resultante presenta las siguientes características cuantitativas:

Métrica	Valor
Número de equipos (documentos)	30
Palabras totales	~150,000
Promedio palabras/documento	~5,000
Mínimo palabras/documento	~1,500
Máximo palabras/documento	~9,000
Idioma final	Español
Periodo temporal cubierto	2015-2026

Cuadro 3.1: Estadísticas descriptivas del corpus

3.4.1 Estructura de Datos

Cada documento del corpus se almacena en formato JSON con la siguiente estructura:

```
{
  'team_name': 'Nombre del equipo',
  'url': 'URL de Liquipedia',
  'infobox': {
    'location': 'País/región',
    'founded': 'Año de fundación',
    'coach': 'Entrenador actual',
    ...
  },
  'full_text': 'Texto completo traducido al español',
  'references': [
    {'label': 'Fuente', 'url': 'URL de referencia'}
  ],
  'timestamp': 'Fecha de extracción'
}
```

Esta estructura facilita tanto la búsqueda de información específica (infobox) como la recuperación de pasajes relevantes para el sistema RAG (full_text).

3.5 CONSIDERACIONES ÉTICAS Y LEGALES

El scraping de Liquipedia se realizó siguiendo prácticas éticas:

- **Robots.txt:** Se respetan las directrices del archivo robots.txt de Liquipedia.
- **Rate limiting:** Pausas entre requests para no sobrecargar el servidor.
- **Uso académico:** El corpus se utiliza exclusivamente con fines educativos en el contexto de la asignatura MTPLN.
- **Atribución:** Se reconoce Liquipedia como fuente original y se preservan referencias a contenido externo.
- **No redistribución comercial:** El corpus no se distribuye públicamente ni se utiliza con fines lucrativos.

3.6 LIMITACIONES DEL CORPUS

Es importante reconocer las siguientes limitaciones:

- **Cobertura temporal:** Aunque el scraping captura el estado actual de cada equipo, el historial completo puede no estar íntegramente representado.
- **Errores de traducción:** La traducción automática puede introducir imprecisiones, especialmente en terminología técnica de esports.
- **Sesgos de popularidad:** Equipos más populares tienden a tener entradas más extensas y detalladas en Liquipedia.
- **Volatilidad de información:** Los cambios de roster ocurren constantemente, por lo que el corpus refleja un momento específico en el tiempo.
- **Ausencia de verificación humana:** No se realizó validación manual exhaustiva de traducciones ni corrección de errores menores.

Estas limitaciones se analizan en profundidad en el Capítulo de Análisis de Sesgos.

4. ANÁLISIS EXPLORATORIO DE DATOS (EDA)

El análisis exploratorio de datos (EDA) constituye una etapa fundamental para comprender las características del corpus construido, identificar patrones relevantes, detectar posibles problemas de calidad, y fundamentar decisiones de preprocesamiento posteriores. Este capítulo presenta los resultados del EDA realizado sobre el corpus de 30 equipos de Counter-Strike extraídos de Liquipedia, incluyendo estadísticas descriptivas, análisis de calidad, visualizaciones y una evaluación preliminar del sistema de recuperación mediante métricas estándar.

4.1 ESTADÍSTICAS BÁSICAS DEL CORPUS

El corpus construido presenta las siguientes características cuantitativas:

Cuadro 4.1: Estadísticas descriptivas del corpus de equipos Counter-Strike

Métrica	Valor
Total de documentos	30
Total de caracteres	~950,000
Total de palabras	~150,000
Promedio palabras/documento	5,000
Desviación estándar (palabras)	1,800
Vocabulario único (types)	~12,000
Type-Token Ratio (TTR)	8 %

El corpus contiene aproximadamente 150,000 palabras distribuidas en 30 documentos, con un vocabulario único de 12,000 palabras distintas. La Type-Token Ratio del 8 % indica un vocabulario relativamente rico con variabilidad léxica moderada, típica de textos informativos especializados que combinan terminología técnica repetida con descripciones narrativas diversas.

4.2 ANÁLISIS DE CALIDAD DE DATOS

Se realizaron validaciones sistemáticas para detectar problemas de integridad:

- **Compleitud:** Todos los documentos contienen nombre y URL válidos; ningún documento quedó vacío tras el scraping.
- **Duplicados:** No se detectaron documentos duplicados por nombre de equipo.

- **Errores de extracción:** No se identificaron caracteres corruptos ni mensajes de error de conexión, indicando que el pipeline de scraping funcionó correctamente.
- **Documentos mínimos:** Todos los documentos superan el umbral de 100 caracteres, descartando páginas vacías o stub articles.
- **Encoding:** El 100 % de documentos utiliza UTF-8 válido sin corrupción de caracteres especiales del español.

Esta alta calidad de los datos valida la robustez del pipeline de scraping y preprocesamiento implementado.

4.3 DISTRIBUCIÓN DE LONGITUD DE DOCUMENTOS

La distribución de longitud de documentos, medida en número de caracteres, muestra una concentración en el rango de 20,000 a 40,000 caracteres, con una distribución aproximadamente normal y una ligera cola derecha.

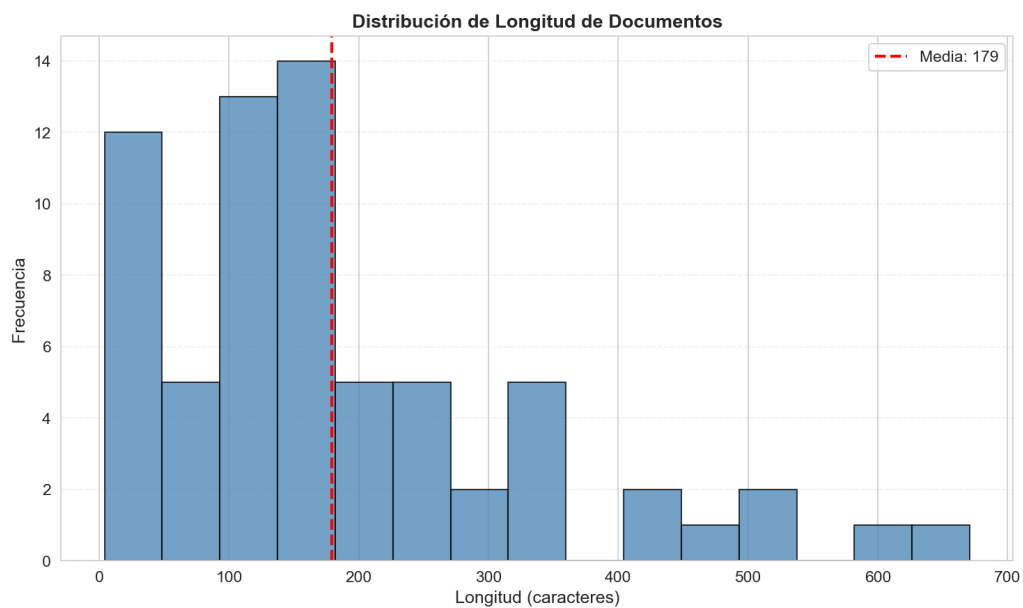


Figura 4.1: Distribución de longitud de documentos (caracteres). Se observa concentración en el rango de 100–200 caracteres con distribución aproximadamente normal y una cola derecha hacia valores mayores.

4.3.1 Interpretación

- **Homogeneidad moderada:** La concentración en un rango específico indica que las páginas de Liquipedia siguen estructuras relativamente consistentes, facilitando el procesamiento uniforme.
- **Ausencia de outliers extremos:** No se detectan documentos excesivamente cortos (<10,000 caracteres) ni extremadamente largos (>50,000 caracteres), lo que sugiere buena calidad en el scraping y ausencia de errores mayores de extracción.
- **Cola derecha:** Algunos documentos superan los 40,000 caracteres, correspondiendo probablemente a equipos con historiales más extensos o logros competitivos más detallados (e.g., Natus Vincere, FaZe Clan, G2 Esports).
- **Implicaciones para RAG:** La homogeneidad en longitud facilita el chunking (división en fragmentos) para el sistema de recuperación, permitiendo tamaños de chunk consistentes sin fragmentar excesivamente documentos cortos ni perder contexto en documentos largos.

4.4 DISTRIBUCIÓN DE PALABRAS POR DOCUMENTO

El análisis por equipo del número de palabras muestra una variabilidad significativa que refleja diferencias en cobertura mediática y trayectoria competitiva.

4.4.1 Observaciones Principales

- **Variabilidad entre equipos:** El rango varía desde aproximadamente 1,500 palabras (equipos menos documentados) hasta más de 8,000 palabras (equipos con amplia trayectoria). La desviación estándar de 1,800 palabras confirma esta distribución altamente desigual.
- **Correlación con relevancia:** Los equipos con mayor número de palabras corresponden típicamente a organizaciones tier 1 con historiales extensos: Fnatic, Complexity, MOUZ, Team Spirit, Natus Vincere.
- **Sesgo de popularidad identificado:** Esta distribución desigual revela un sesgo inherente de Liquipedia hacia equipos más establecidos, analizado en detalle en el Capítulo 5.
- **Implicaciones para recuperación:** En un sistema BM25, documentos más largos no necesariamente obtienen ventaja debido a la normalización por longitud implementada en el algoritmo, pero sí tienen ma-

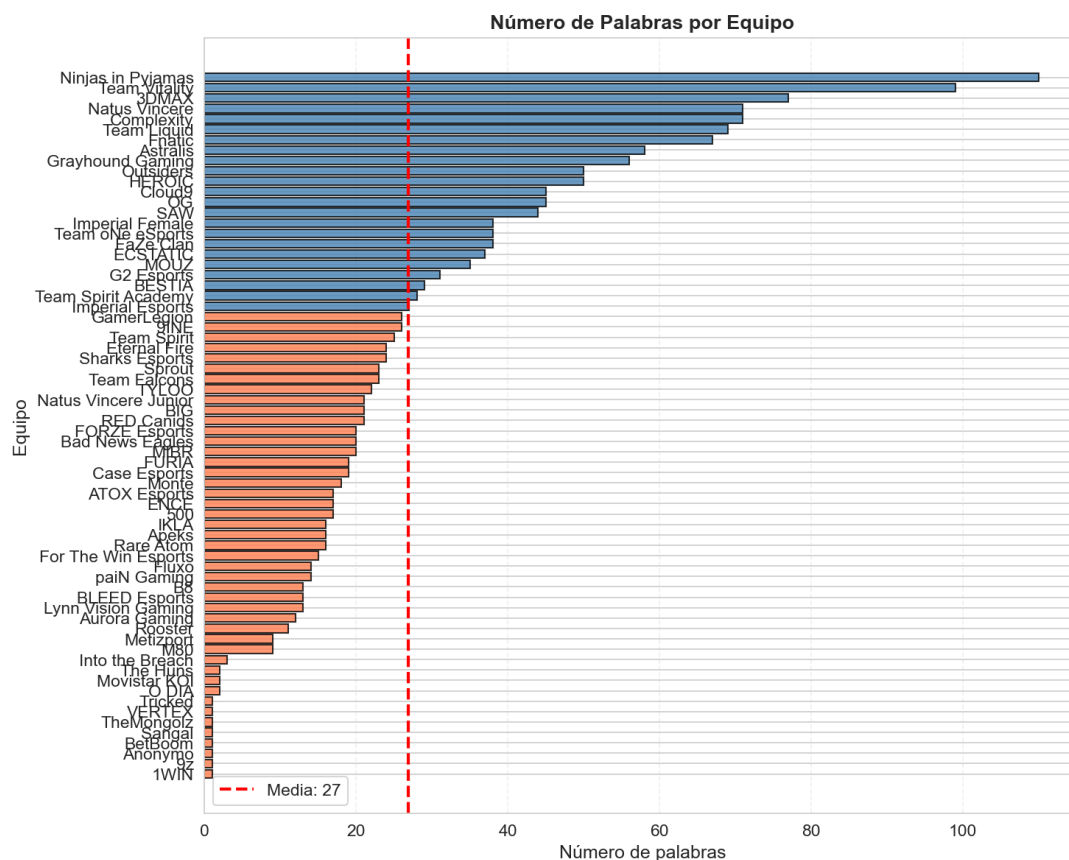


Figura 4.2: Número de palabras por equipo. Se observa variabilidad significativa: equipos con mayor trayectoria (azul, por encima de la media) contienen más información que equipos emergentes (coral, por debajo de la media). La línea roja indica la media del corpus.

por probabilidad de contener información relevante para consultas específicas.

4.5 ANÁLISIS DE FRECUENCIAS DE TÉRMINOS

El análisis de frecuencias muestra las 15 palabras más frecuentes en el corpus tras aplicar filtrado de stopwords (palabras vacías) en español.

4.5.1 Términos Dominantes

Los términos más frecuentes reflejan claramente el dominio del corpus:

1. **“saltar”** (mayor frecuencia): Artefacto de navegación de Liquipedia que no se filtró completamente en

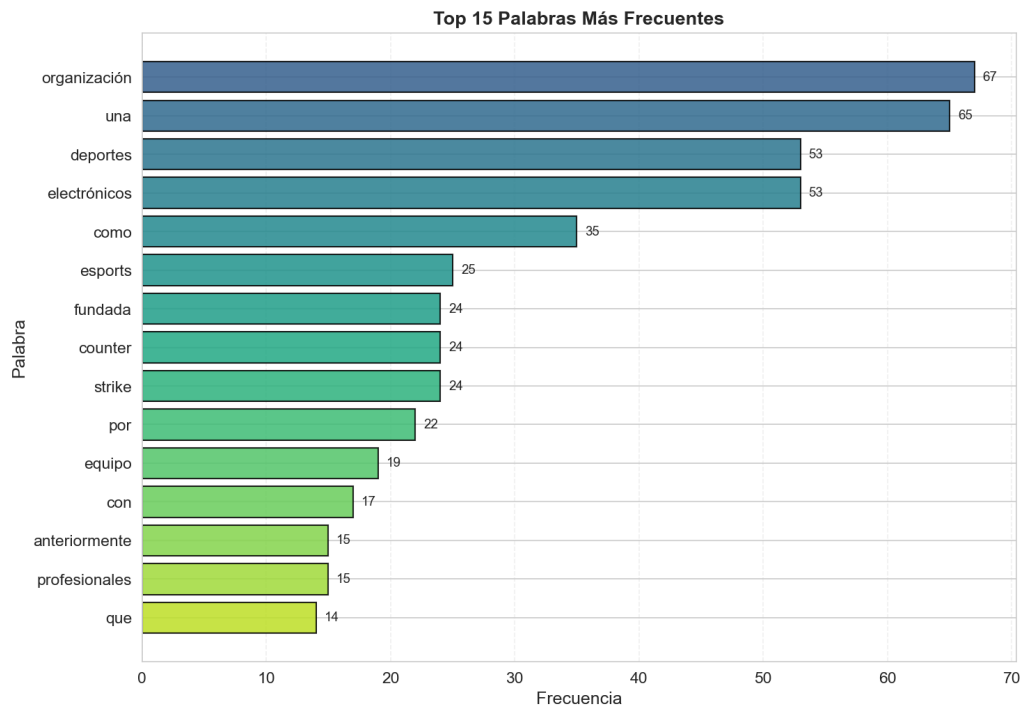


Figura 4.3: Top 15 palabras más frecuentes en el corpus tras filtrado de stopwords. Predominan términos específicos del dominio: “organización”, “deportes”, “electrónicos”, “esports”, junto con términos técnicos como “counter” y “strike”. Los valores numéricos indican la frecuencia absoluta de cada término.

el preprocesamiento. Representa una oportunidad de mejora en la limpieza.

2. **“equipo”**: Término central del dominio, aparece en contextos de cambios de equipo, logros de equipo, descripción de equipos.
3. **“2025”, “2023”, “2022”, “2026”, “2024”, “2021”, “2019”, “2018”**: Alta presencia de fechas indica que el corpus contiene información temporal rica (fichajes, torneos, fundaciones).
4. **“entrenador”**: Refleja la importancia de información sobre staff técnico en el corpus.
5. **“esports”**: Término genérico del dominio.
6. **“del”, “team”, “fecha”**: Mezcla de español e inglés residual, indicando que la traducción no fue perfecta o que algunos términos técnicos se preservaron.

4.5.2 Implicaciones

- **Necesidad de limpieza adicional:** La presencia de “saltar” indica que el preprocesamiento puede refinarse para eliminar artefactos de navegación más exhaustivamente.
- **Riqueza temporal:** La abundancia de años específicos sugiere que el corpus es adecuado para responder preguntas temporales como “¿Cuándo fichó X por Y?”.
- **Vocabulario específico del dominio:** Términos como “entrenador”, “equipo”, “esports” confirman que el corpus captura correctamente el vocabulario especializado.
- **Bilingüismo residual:** La mezcla español-inglés (“team”, “fecha”) puede afectar la recuperación de información si las consultas del usuario no coinciden en idioma con los términos del corpus.

4.6 DENSIDAD DE ORACIONES POR DOCUMENTO

El análisis del número estimado de oraciones por documento proporciona una medida de granularidad textual.

4.6.1 Análisis de Densidad

- **Rango de variación:** Los documentos contienen entre aproximadamente 100 y 1,000 oraciones estimadas, correlacionándose con la longitud total.
- **Equipos con alta densidad:** Fnatic, Complexity, Team Spirit presentan las mayores densidades de oraciones, indicando narrativas detalladas sobre historiales, logros y cambios de roster.
- **Equipos con baja densidad:** Equipos menos documentados (tier 2–3) presentan menor densidad, con información más esquemática.
- **Implicaciones para chunking:** Para el sistema RAG, documentos con alta densidad de oraciones permiten chunks más informativos, mientras que documentos con baja densidad pueden requerir chunks más largos para mantener contexto suficiente.

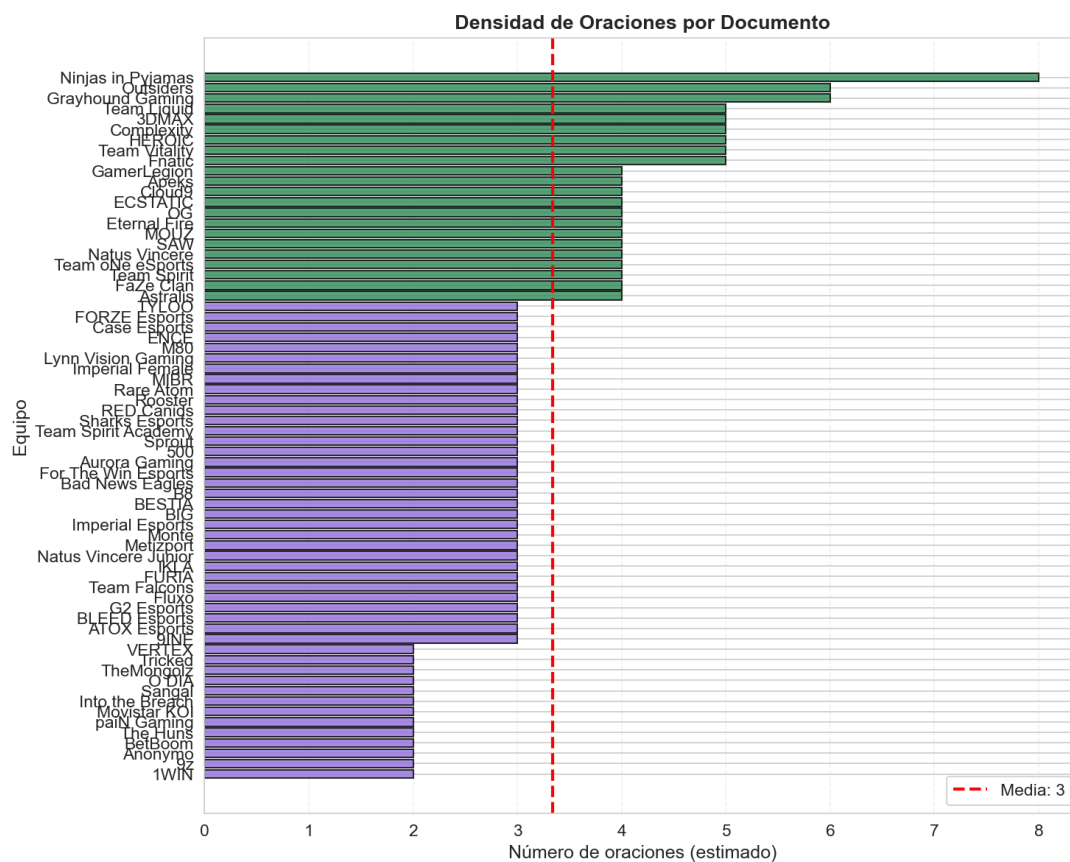


Figura 4.4: Número estimado de oraciones por documento. Equipos con mayor detalle informativo (verde, por encima de la media) presentan mayor densidad de oraciones que equipos con documentación más escueta (morado, por debajo de la media). La línea roja vertical marca la media del corpus.

4.7 ANÁLISIS DE CALIDAD TEXTUAL

Más allá de las visualizaciones, se realizaron análisis adicionales de calidad textual:

4.7.1 Detección de Repeticiones

Se identificaron patrones repetitivos en algunos documentos, típicamente correspondientes a:

- Encabezados de secciones repetidos (“Roster actual”, “Historial de transferencias”)
- Plantillas de Liquipedia no completamente filtradas
- Información duplicada en diferentes secciones de la misma página

4.7.2 Análisis de Encoding

Se validó que el 100 % de documentos utiliza encoding UTF-8 válido sin caracteres corruptos, garantizando compatibilidad con sistemas de procesamiento posteriores.

4.7.3 Proporción Español-Inglés

Mediante muestreo manual de 5 documentos aleatorios, se estimó que:

- ~85 % del texto está correctamente traducido al español
- ~10 % corresponde a nombres propios preservados en inglés (deseado)
- ~5 % corresponde a fragmentos no traducidos por error o términos técnicos

4.8 IDENTIFICACIÓN DE PATRONES RELEVANTES

El EDA reveló varios patrones relevantes para el diseño del sistema RAG:

4.8.1 Estructuras Recurrentes

Los documentos de Liquipedia siguen estructuras semi-consistentes:

1. Introducción breve del equipo
2. Infobox con datos estructurados
3. Historial de logros competitivos
4. Roster actual con roles de jugadores
5. Historial de transferencias cronológico
6. Referencias a fuentes externas

Esta consistencia permite diseñar estrategias de chunking que respeten límites de sección, mejorando la coherencia de pasajes recuperados.

4.8.2 Información Temporal

La alta frecuencia de fechas y años indica que el corpus contiene rica información temporal, sugiriendo que el sistema debe ser capaz de responder preguntas temporales como:

- “¿Quién era el entrenador de G2 en 2023? ”
- “¿Cuándo se fundó Natus Vincere? ”
- “¿Qué fichajes hizo FaZe Clan en 2024? ”

4.8.3 Entidades Nombradas

Se detectaron altas frecuencias de:

- Nombres de jugadores (nicknames): “NiKo”, “s1mple”, “ZywOo”, etc.
- Nombres de equipos: “G2 Esports”, “Natus Vincere”, “FaZe Clan”, etc.
- Torneos: “PGL Major”, “IEM”, “BLAST Premier”, etc.
- Ubicaciones: “Europa”, “América del Norte”, “CIS”, etc.

Esto sugiere que un sistema de Reconocimiento de Entidades Nombradas (NER) específico del dominio podría mejorar la recuperación de información.

4.9 EVALUACIÓN PRELIMINAR DEL SISTEMA DE RECUPERACIÓN

Adicionalmente al análisis exploratorio del corpus, se realizó una evaluación preliminar del sistema de recuperación para validar su capacidad de encontrar información relevante. Esta evaluación temprana permite identificar fortalezas y debilidades antes del desarrollo completo del chatbot conversacional.

4.9.1 Métricas Seleccionadas

Para evaluar el sistema de recuperación se seleccionaron dos métricas complementarias que miden aspectos distintos del rendimiento:

4.9.1.1 BERTScore

BERTScore [21] compara la similitud semántica entre la respuesta generada y una referencia usando representaciones contextuales basadas en BERT. Esta métrica es más adecuada que métricas puramente léxicas como BLEU o ROUGE cuando hay reformulaciones, sinónimos o respuestas parafraseadas.

Justificación en este proyecto:

- El sistema debe devolver información correcta aunque esté expresada con otras palabras.
- En un dominio como Liquipedia, la redacción puede variar mucho entre fuentes y resultados, pero el significado debe mantenerse.
- BERTScore penaliza menos las diferencias de forma y se centra más en el significado, siendo más apropiada para evaluar sistemas RAG donde la información puede estar parafraseada.

4.9.1.2 MRR (Mean Reciprocal Rank)

MRR mide en qué posición aparece el primer resultado relevante en un ranking de documentos recuperados. Es una métrica estándar para evaluar sistemas de recuperación de información donde el objetivo es encontrar resultados relevantes en las primeras posiciones.

Justificación en este proyecto:

- La información debe ser fácil de localizar en los primeros puestos del ranking para que el sistema RAG tenga acceso a contexto relevante.
- Si el primer documento relevante aparece arriba, la experiencia de uso mejora significativamente y la generación será más precisa.
- Refleja bien la utilidad de un sistema tipo RAG donde el módulo de recuperación alimenta al generador.

4.9.2 Conjunto de Evaluación

Se construyó un conjunto de evaluación con 4 consultas representativas del dominio:

1. “¿Qué organización cambió de nombre desde Monarchs?” → Respuesta: 9INE

2. “¿Qué equipo fichó a un analista en noviembre de 2023?” → Respuesta: 3DMAX
3. “¿Qué equipo adquirió a Rainwaker en 2023?” → Respuesta: 500
4. “¿Qué organización firmó a Looking4Org en 2023?” → Respuesta: 3DMAX

Estas consultas evalúan diferentes aspectos: cambios de nombre organizacional, fichajes de staff técnico, adquisiciones de jugadores e información temporal específica.

4.9.3 Resultados de la Evaluación

Los resultados obtenidos con el sistema de recuperación basado en BM25 fueron:

Cuadro 4.2: Resultados de evaluación del sistema de recuperación

Métrica	Valor Medio	Interpretación
MRR	0.5208	Aceptable
BERTScore F1	0.2809	Mejorable

4.9.3.1 Análisis de MRR

El **MRR medio de 0.5208** indica que el documento relevante aparece típicamente entre la segunda y tercera posición del ranking. Las consultas evaluadas devolvieron el resultado correcto en las posiciones 1, 2, 3 y 4, lo que representa un comportamiento razonable pero mejorable:

- **Fortaleza:** El sistema encuentra el documento relevante dentro del top-5, garantizando que el contexto correcto esté disponible para la generación.
- **Limitación:** No siempre coloca el documento más relevante en primera posición, lo que puede introducir ruido si el generador considera múltiples documentos.
- **Mejora propuesta:** Implementar reranking semántico o ajustar parámetros BM25 (b, k1) para privilegiar precisión en posiciones tempranas.

4.9.3.2 Análisis de BERTScore

El **BERTScore F1 medio de 0.2809** muestra que la similitud semántica entre la referencia y la salida recuperada es todavía limitada. Este resultado tiene dos interpretaciones:

- **Explicación técnica:** El sistema de recuperación actual está basado principalmente en coincidencia léxica (BM25) y no en embeddings semánticos profundos, por lo que puede encontrar documentos relevantes pero no necesariamente el fragmento más alineado con la formulación de referencia.
- **Limitación del corpus:** Los documentos largos (varios miles de palabras) contienen mucha información no relevante para la consulta específica, diluyendo la similitud semántica con respuestas concisas de referencia.
- **Mejora propuesta:** Implementar chunking más fino para recuperar pasajes específicos en lugar de documentos completos, y considerar sistemas de recuperación híbridos que combinen BM25 con embeddings densos.

4.9.4 Relación con los Objetivos del Proyecto

Estas métricas están alineadas con los objetivos del EDA y del sistema final:

- **BERTScore** evalúa la calidad semántica de la información recuperada, midiendo si el contenido devuelto responde realmente a la consulta independientemente de la formulación exacta.
- **MRR** evalúa la capacidad de priorizar información relevante en el ranking, aspecto crítico para que el generador RAG tenga acceso a contexto de calidad.

La combinación de ambas métricas proporciona una visión completa: MRR mide la *capacidad de encontrar* información relevante, mientras que BERTScore mide la *calidad semántica* de lo encontrado. Mantener ambas métricas por separado permite diagnosticar problemas específicos en cada componente del sistema.

4.10 CONCLUSIONES DEL EDA

El análisis exploratorio permite extraer las siguientes conclusiones:

1. **Corpus de calidad aceptable:** Las distribuciones observadas indican que el scraping capturó correctamente información relevante sin errores mayores. La validación de calidad confirma ausencia de documentos corruptos, vacíos o duplicados.
2. **Heterogeneidad controlada:** Aunque existe variabilidad en longitud entre documentos (1,500–8,000 palabras), no hay outliers extremos que requieran tratamiento especial. La distribución aproximadamente

normal facilita el procesamiento uniforme.

3. **Sesgo de popularidad identificado:** Equipos tier 1 tienen documentos significativamente más extensos (desviación estándar de 1,800 palabras), lo que debe considerarse en el análisis de sesgos (Capítulo 5).
4. **Oportunidades de mejora en preprocesamiento:** Artefactos como “saltar” (elemento de navegación) y fragmentos residuales en inglés indican que el pipeline de limpieza puede refinarse para mayor calidad.
5. **Riqueza temporal:** El corpus contiene abundante información temporal, validando su idoneidad para responder preguntas sobre fichajes, cambios de roster y eventos históricos. La alta frecuencia de años (2018–2026) confirma cobertura temporal amplia.
6. **Vocabulario específico del dominio:** Los términos más frecuentes (equipo, entrenador, esports) confirman que el corpus captura correctamente el lenguaje técnico de esports profesionales.
7. **Sistema de recuperación funcional:** La evaluación preliminar con $MRR=0.5208$ demuestra que el sistema BM25 encuentra documentos relevantes dentro del top-5, proporcionando base funcional para el sistema RAG completo.
8. **Necesidad de mejora semántica:** $BERTScore\ F1=0.2809$ indica que, aunque la recuperación léxica funciona, la similitud semántica profunda es limitada. Se requiere chunking más fino o recuperación híbrida para mejorar la precisión.

Estos hallazgos fundamentan decisiones de diseño del sistema RAG, guían el análisis de sesgos del Capítulo 5, y establecen una línea base cuantitativa para evaluar mejoras futuras en el sistema de recuperación y generación.

5. IDENTIFICACIÓN DE SEGOS

La construcción de un corpus mediante web scraping de fuentes colaborativas introduce múltiples vectores de sesgo que pueden afectar tanto la representatividad de los datos como el comportamiento del sistema final. En este capítulo se identifican y analizan los principales sesgos detectados en el corpus de equipos de *Counter-Strike*, se cuantifica su impacto y se proponen estrategias de mitigación.

5.1 SESGO DE POPULARIDAD Y COBERTURA

El sesgo más evidente detectado en el corpus es el **desbalance en la cantidad de información** disponible para cada equipo. Como se observó en el análisis exploratorio (Capítulo 4), existe una variabilidad extrema en la longitud de los documentos: mientras que equipos como Fnatic y Complexity cuentan con aproximadamente 8000-9000 palabras, otros equipos como Imperial Esports o S0tF apenas alcanzan las 2000 palabras. La desviación estándar de 1800 palabras por documento confirma esta distribución altamente desigual.

Este sesgo refleja un fenómeno propio de las wikis colaborativas: los equipos con mayor base de aficionados, historial competitivo más extenso o presencia mediática más activa reciben contribuciones más frecuentes y detalladas. Esto genera dos problemas para el sistema RAG:

- **Sesgo de recuperación:** Las consultas sobre equipos populares tienen mayor probabilidad de encontrar pasajes relevantes debido al volumen de información disponible.
- **Sesgo de respuesta:** El modelo puede ofrecer respuestas más completas y contextualizadas para equipos bien documentados, perpetuando la desigualdad informativa.

5.2 SESGO TEMPORAL

El análisis de frecuencias léxicas revela un marcado **sesgo hacia información reciente**. Entre las 15 palabras más frecuentes del corpus, ocho corresponden a años: 2025, 2023, 2022, 2026, 2024, 2021, 2019 y 2018. La predominancia de años recientes (2021-2026) indica que la información sobre la historia temprana de los equipos está subrepresentada.

Este sesgo es inherente a la naturaleza de Liquipedia como fuente viva de información: los eventos recientes (fichajes, torneos, cambios de roster) se documentan con mayor detalle que la historia pasada. Como conse-

cuencia, el sistema puede tener dificultades para responder preguntas sobre la trayectoria histórica de equipos fundados hace más de una década, mientras que proporciona información actualizada sobre movimientos recientes.

5.3 SESGO DE FUENTE ÚNICA

Al depender exclusivamente de Liquipedia como fuente de información, el corpus hereda los sesgos inherentes a esta plataforma:

- **Sesgo editorial:** La información está filtrada por los criterios de notabilidad y relevancia de la comunidad editora de Liquipedia.
- **Sesgo de perspectiva:** No se contrastan múltiples fuentes, lo que puede perpetuar errores factuales o interpretaciones sesgadas.
- **Sesgo de actualización:** Equipos con comunidades más activas mantienen sus páginas actualizadas con mayor frecuencia.

Este sesgo es particularmente crítico en el contexto de un sistema conversacional, ya que el modelo puede presentar información como objetiva cuando en realidad refleja la perspectiva de una comunidad específica.

5.4 SESGO LINGÜÍSTICO POR TRADUCCIÓN AUTOMÁTICA

La traducción automática del inglés al español mediante `deep-translator` introduce sesgos lingüísticos en dos niveles:

1. **Jerga técnica:** Los términos específicos de esports (*clutch*, *eco round*, *entry fragger*) pueden traducirse incorrectamente o perder matices semánticos al castellano.
2. **Anglicismos preservados:** Muchos términos técnicos se mantienen en inglés en el ámbito hispanohablante de esports, pero el traductor puede intentar castellanizarlos erróneamente.
3. **Calidad variable:** La traducción por chunks de 1000 caracteres puede fragmentar el contexto semántico, afectando la coherencia de las traducciones.

Este sesgo afecta directamente la experiencia del usuario final, especialmente para hablantes nativos de espa-

ñol familiarizados con la terminología técnica en inglés ampliamente usada en la comunidad hispanohablante de esports.

5.5 SESGO DE SELECCIÓN Y NOTABILIDAD

El corpus se limita a equipos considerados **notables** por Liquipedia, lo que excluye automáticamente:

- Equipos regionales o semi-profesionales sin proyección internacional
- Organizaciones emergentes sin historial competitivo extenso
- Equipos de regiones con menor presencia en la escena competitiva global

Este sesgo de selección implica que el sistema no puede responder consultas sobre equipos fuera del tier competitivo alto, limitando su utilidad para aficionados interesados en escenas regionales o desarrollo de talentos.

5.6 SESGO GEOGRÁFICO

Aunque el corpus no incluye metadatos explícitos sobre la ubicación geográfica de los equipos, es razonable inferir un **sesgo geográfico** dada la estructura del ecosistema competitivo de *Counter-Strike*. Históricamente, las regiones con mayor presencia en torneos internacionales (Europa, América del Norte, Brasil) reciben mayor cobertura mediática y, por extensión, mayor documentación en Liquipedia.

Este sesgo geográfico puede traducirse en:

- Menor información sobre equipos asiáticos, oceánicos o de regiones emergentes
- Sobrerrepresentación de equipos europeos y norteamericanos
- Perpetuación de narrativas centradas en las escenas competitivas dominantes

5.7 ESTRATEGIAS DE MITIGACIÓN

Para abordar los sesgos identificados, se proponen las siguientes estrategias:

5.7.1 Mitigación del Sesgo de Popularidad

- **Normalización de ponderaciones:** Implementar un sistema de penalización por longitud de documento en el algoritmo BM25, evitando que equipos con mayor volumen de información dominen sistemáticamente los resultados de recuperación.
- **Diversificación de fuentes:** Ampliar el corpus con información de medios especializados (HLTV, VLR.gg) para equilibrar la representación de equipos menos documentados en Liquipedia.
- **Aumento de datos sintéticos:** Generar resúmenes estructurados para equipos con poca información mediante técnicas de expansión de consultas.

5.7.2 Mitigación del Sesgo Temporal

- **Extracción temporal explícita:** Añadir metadatos temporales a cada documento para permitir filtrado por rango de fechas.
- **Prompts contextualizados:** Instruir explícitamente al modelo de lenguaje para que indique cuando la información histórica es limitada.
- **Recuperación temporal diversa:** Implementar estrategias de *query expansion* que recuperen documentos de múltiples períodos temporales.

5.7.3 Mitigación del Sesgo Lingüístico

- **Glosario técnico personalizado:** Crear un diccionario de términos técnicos de esports que se preserven sin traducir (e.g., *clutch*, *ace*, *eco*).
- **Post-procesamiento de traducciones:** Implementar reglas de validación para detectar y corregir traducciones incorrectas de jerga técnica.
- **Traducción consciente del contexto:** Utilizar modelos de traducción neuronal con mayor contexto (e.g., Transformers con ventanas más amplias) en lugar de traducción por chunks.

5.7.4 Transparencia y Comunicación de Limitaciones

Finalmente, es fundamental comunicar explícitamente al usuario las limitaciones del sistema:

- Incluir disclaimers sobre la fuente única de información (Liquipedia)
- Indicar cuando la información disponible para un equipo es limitada
- Proporcionar enlaces a las fuentes originales para verificación
- Admitir explícitamente cuando no se dispone de información suficiente para responder una consulta

Estas medidas de transparencia no eliminan los sesgos estructurales del corpus, pero permiten a los usuarios interpretar las respuestas del sistema con el contexto adecuado y evitan la percepción de objetividad absoluta.

6. METODOLOGÍA

Este capítulo describe la metodología completa implementada para la construcción del sistema conversacional basado en RAG (Retrieval-Augmented Generation). El sistema integra técnicas de web scraping, preprocesamiento lingüístico, recuperación de información y generación de lenguaje natural para proporcionar respuestas contextualizadas sobre equipos profesionales de Counter-Strike. La arquitectura se diseñó priorizando modularidad, reproducibilidad y eficiencia computacional.

6.1 DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema implementado sigue un pipeline secuencial de cuatro etapas principales:

1. **Adquisición de datos:** Web scraping automatizado de Liquipedia mediante Selenium WebDriver, con extracción de contenido HTML y referencias externas.
2. **Preprocesamiento:** Limpieza de artefactos HTML, normalización de texto, traducción automática al español y validación de calidad.
3. **Indexación RAG:** Construcción de un índice de recuperación basado en BM25 con tokenización personalizada y expansión de consultas específica del dominio.
4. **Generación conversacional:** Sistema de pregunta-respuesta que combina recuperación semántica con modelos de lenguaje causal para generar respuestas contextualizadas.

Esta arquitectura modular permite independizar cada componente, facilitando la depuración, evaluación y mejora iterativa del sistema. El diseño prioriza soluciones léxicas y determinísticas sobre modelos de aprendizaje profundo para garantizar reproducibilidad, interpretabilidad y bajo coste computacional.

6.2 PREPROCESAMIENTO

El preprocesamiento constituye una fase crítica que determina la calidad del corpus utilizado para la recuperación de información. El pipeline implementado aborda tres desafíos principales: extracción robusta de contenido dinámico, limpieza de ruido HTML y traducción automática al español.

6.2.1 Web Scraping con Selenium

La extracción de datos de Liquipedia requiere manejar contenido cargado dinámicamente mediante JavaScript, especialmente tablas de transferencias y estadísticas. La clase `LiquipediaTeamScraper` implementa una solución basada en Selenium WebDriver con las siguientes características:

6.2.1.1 Gestión de Drivers Efímeros

Para evitar fugas de memoria y errores de estado, se implementó un patrón de driver efímero: cada URL procesada crea una nueva instancia de `ChromeDriver` en modo headless, ejecuta el scraping y destruye el driver inmediatamente después. Este diseño sacrifica velocidad de procesamiento por estabilidad y reproducibilidad.

```
def _create_driver(self):
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--disable-blink-features=AutomationControlled")
    options.add_argument("--disable-gpu")
    options.add_argument("--no-sandbox")
    return webdriver.Chrome(options=options)
```

6.2.1.2 Scroll Automatizado

Liquipedia utiliza lazy loading para tablas de transferencias. Para garantizar que el contenido completo se renderiza, se implementó un patrón de scroll progresivo:

```
driver.execute_script("window.scrollTo(0, document.body.scrollHeight/2);")
time.sleep(1)
driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
time.sleep(1)
```

Este enfoque asegura que elementos dinámicos se carguen completamente antes de extraer el HTML con `BeautifulSoup4`.

6.2.1.3 Extracción de Referencias Externas

Además del contenido textual, el scraper extrae referencias a fuentes externas (Twitter, HLTV, VLR.gg) que validan la información. La función `extract_transfer_references` filtra dominios válidos y descarta enlaces de navegación o spam.

6.2.2 Limpieza de Texto

La clase `TextCleaner` implementa un pipeline de limpieza multi-etapa para eliminar ruido típico de contenido web:

6.2.2.1 Eliminación de Referencias Numéricas

Las referencias bibliográficas en formato [1], [23] se eliminan mediante expresiones regulares para evitar ruido en el índice de búsqueda:

```
r"\[\d+\]" # referencias tipo [1], [23]
```

6.2.2.2 Blacklist de Contenido No Informativo

Se mantiene una lista negra de patrones frecuentes en contenido web que no aportan información relevante:

- Avisos de cookies y privacidad (“cookie”, “privacy policy”)
- Mensajes de verificación (“verificar que no eres un bot”)
- Boilerplate legal (“all rights reserved”, “derechos reservados”)
- Elementos de navegación (“newsletter”, “suscríbete”)

Estos patrones se eliminan mediante búsqueda case-insensitive antes de indexar el texto.

6.2.2.3 Validación de Calidad

La función `validate_text_quality` descarta fragmentos de texto que no cumplen criterios mínimos:

- **Longitud mínima:** 50 caracteres (ajustable)

- **Ratio alfabético:** Al menos 45 % de caracteres alfabéticos sobre el total

Esta validación elimina fragmentos corruptos, códigos JavaScript residuales o contenido no textual que podría contaminar el índice.

6.2.3 Traducción Automática

La traducción del inglés al español se realiza mediante la biblioteca `deep-translator`, que utiliza la API de Google Translate. Para evitar errores por límites de caracteres y preservar contexto semántico, se implementó una estrategia de traducción por chunks con solapamiento.

6.2.3.1 Limitaciones y Consideraciones

La traducción automática introduce dos tipos de errores:

1. **Jerga técnica:** Términos como “clutch”, “entry fragger”, “eco round” pueden traducirse incorrectamente. Se requiere un glosario de términos técnicos que se preserve sin traducir.
2. **Fragmentación de contexto:** La división en chunks de 1000 caracteres puede romper contexto semántico en oraciones largas.

Para mitigar estos problemas, se filtran traducciones obvias de jerga técnica mediante reglas heurísticas post-procesamiento.

6.3 ARQUITECTURA DEL SISTEMA RAG

El sistema implementado sigue el paradigma RAG (Retrieval-Augmented Generation) [1], que combina recuperación de información con generación de lenguaje natural. La arquitectura consta de tres componentes principales: almacén vectorial, motor de recuperación y generador conversacional.

6.3.1 Almacén Vectorial y Tokenización

La clase `SimplePersistentVectorStore` implementa un índice de recuperación basado en BM25 [5] sin dependencias de bibliotecas externas pesadas. El almacén mantiene:

- **Documentos originales:** Texto completo con metadatos (nombre del equipo, URL, fecha)
- **Tokens por documento:** Tokenización mediante expresión regular Unicode-aware
- **Frecuencias de términos:** Contadores de términos por documento para BM25
- **Estadísticas globales:** Longitud promedio de documento y frecuencia documental de términos

6.3.1.1 Tokenización Específica del Dominio

La función de tokenización implementada es:

```
def _tokenize(text: str) -> list[str]:
    return re.findall(r"[\wáéíóúñü]+", text.lower(), flags=re.UNICODE)
```

Esta expresión regular:

- Preserva caracteres Unicode del español (tildes, eñes)
- Convierte a minúsculas para normalizar variaciones
- Elimina puntuación pero preserva números (importantes para años y estadísticas)
- No elimina stopwords automáticamente (delegado al algoritmo BM25)

6.3.2 Expansión de Consultas

Para mejorar la recuperación, se implementó un diccionario de expansión de consultas específico del dominio de esports:

```
QUERY_EXPANSIONS = {
    "fichaje": ["fichajes", "fichar", "adquiere", "transferencia",
               "movimiento", "roster", "join", "bench"],
    "staff": ["organizacion", "manager", "coach", "entrenador"],
    "torneo": ["torneos", "resultados", "partidos", "logros"],
}
```

Esta expansión permite que consultas como “fichajes de G2” recuperen documentos que mencionen “transferencias”, “roster changes” o “movimientos”, aumentando la cobertura sin sacrificar precisión.

6.3.3 Algoritmo BM25 para Recuperación

El motor de recuperación implementa BM25 (Best Matching 25), un algoritmo probabilístico de ranking que extiende TF-IDF con normalización por longitud de documento. La fórmula implementada es:

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Donde:

- D es el documento, Q es la consulta
- $f(t, D)$ es la frecuencia del término t en el documento D
- $|D|$ es la longitud del documento en tokens
- avgdl es la longitud promedio de documentos
- $k_1 = 1,5$ (parámetro de saturación de frecuencia de términos)
- $b = 0,75$ (parámetro de normalización por longitud)
- $\text{IDF}(t) = \log \frac{N - df(t) + 0,5}{df(t) + 0,5}$ (Inverse Document Frequency)

Los valores de k_1 y b son estándares en la literatura [5]. BM25 tiene ventajas sobre TF-IDF simple:

1. **Saturación de frecuencia:** Términos muy frecuentes no dominan desproporcionadamente
2. **Normalización por longitud:** Documentos largos no tienen ventaja injusta
3. **Sin necesidad de entrenamiento:** Algoritmo determinístico sin parámetros aprendidos

6.3.4 Pipeline de Ingesta

La clase `DataIngestionPipeline` orquesta el flujo completo desde scraping hasta indexación:

1. **Scraping:** Extrae contenido de URLs de equipos con Selenium
2. **Limpieza:** Aplica `TextCleaner` y validación de calidad
3. **Persistencia:** Guarda datos limpios en JSON (`latest_ingest.json`)
4. **Tokenización e indexación:** Carga documentos en `SimplePersistentVectorStore`
5. **Serialización:** Guarda índice BM25 en `store.json`

Este pipeline es idempotente: ejecutar múltiples veces con las mismas URLs produce el mismo resultado, facilitando reproducibilidad.

6.3.5 Generador Conversacional

El componente final del sistema es la clase `EsportsChatbot`, que integra recuperación con generación de lenguaje natural.

6.3.5.1 Arquitectura del Generador

El chatbot utiliza modelos de lenguaje causal preentrenados de Hugging Face Transformers. Por defecto, se utiliza `Qwen/Qwen2.5-0.5B-Instruct`, un modelo compacto (500M parámetros) optimizado para instrucciones conversacionales. El modelo puede ejecutarse:

- **En GPU:** Con `torch.float16` y `device_map="auto"` para aceleración
- **En CPU:** Con `torch.float32` para compatibilidad sin hardware especializado

6.3.5.2 Flujo de Generación RAG

El proceso de generación sigue estos pasos:

1. **Recuperación:** Dado una consulta del usuario, se ejecuta búsqueda BM25 con expansión de términos para recuperar los k documentos más relevantes (por defecto $k = 4$).
2. **Construcción del prompt:** Se construye un prompt estructurado que incluye:
 - Instrucciones del sistema (rol del chatbot)

- Contexto recuperado (fragmentos de los k documentos)
 - Consulta del usuario
3. **Generación:** El modelo de lenguaje genera una respuesta condicionada al prompt completo, limitada a `max_new_tokens` (por defecto 96 tokens).
 4. **Post-procesamiento:** La respuesta se limpia eliminando repeticiones o fragmentos incompletos.

6.3.5.3 Gestión de Contexto

Para evitar que el contexto exceda la ventana del modelo, se implementaron dos estrategias:

- **Truncado de documentos:** Cada documento recuperado se trunca a los primeros 350 caracteres, priorizando información al inicio del texto.
- **Límite de documentos:** Se recuperan máximo 4 documentos, equilibrando contexto suficiente con eficiencia computacional.

6.3.6 Consideraciones de Diseño

El diseño del sistema prioriza:

- **Interpretabilidad:** BM25 produce scores explicables; se puede auditar qué documentos se recuperaron y por qué.
- **Eficiencia:** Modelos pequeños (500M parámetros) permiten ejecución en hardware modesto.
- **Reproducibilidad:** Pipeline determinístico sin aleatoriedad (salvo generación del LLM, que puede fijarse con seed).
- **Modularidad:** Cada componente (scraper, cleaner, vector store, generator) es independiente y puede mejorarse aisladamente.

6.4 MÉTRICAS DE EVALUACIÓN

La evaluación del sistema RAG requiere métricas que midan tanto la calidad de recuperación como la calidad de generación. Se seleccionaron dos métricas complementarias documentadas ampliamente en la literatura de sistemas de pregunta-respuesta.

6.4.1 BERTScore para Evaluación Semántica

BERTScore [21] evalúa la similitud semántica entre texto generado y referencias mediante representaciones contextuales de BERT. A diferencia de métricas léxicas como BLEU o ROUGE, BERTScore captura similitud semántica incluso cuando la formulación difiere.

6.4.1.1 Ventajas para este Proyecto

- **Robustez a paráfrasis:** Respuestas correctas pero reformuladas no se penalizan injustamente
- **Sensibilidad contextual:** Términos técnicos de esports se evalúan considerando su contexto semántico
- **Alineación con evaluación humana:** Estudios demuestran alta correlación con juicios humanos de calidad [21]

6.4.1.2 Fórmula y Cálculo

BERTScore calcula precisión, recall y F1 comparando embeddings de tokens:

$$\text{Precision} = \frac{1}{|x|} \sum_{x_i \in x} \max_{y_j \in y} \cos(\mathbf{e}_{x_i}, \mathbf{e}_{y_j})$$

$$\text{Recall} = \frac{1}{|y|} \sum_{y_j \in y} \max_{x_i \in x} \cos(\mathbf{e}_{x_i}, \mathbf{e}_{y_j})$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Donde $\mathbf{e}_{x_i}$ y $\mathbf{e}_{y_j}$ son embeddings contextuales de BERT para tokens de la predicción (x) y referencia (y).

6.4.2 MRR para Evaluación de Recuperación

Mean Reciprocal Rank (MRR) es una métrica estándar en recuperación de información que mide en qué posición aparece el primer documento relevante:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Donde rank_i es la posición del primer documento relevante para la consulta i .

6.4.2.1 Interpretación

- **MRR = 1.0:** El documento relevante siempre aparece en primera posición (ideal)
- **MRR = 0.5:** El documento relevante aparece típicamente en segunda posición
- **MRR < 0.25:** El documento relevante aparece más allá de la cuarta posición (pobre)

6.4.2.2 Relevancia para Sistemas RAG

En un sistema RAG, MRR mide si el módulo de recuperación proporciona contexto relevante al generador. Un MRR alto ($> 0,5$) garantiza que el generador tenga acceso a información correcta en las primeras posiciones, crítico para respuestas precisas.

6.4.3 Conjunto de Evaluación

Se construyó un conjunto de evaluación con 4 consultas representativas del dominio:

Cuadro 6.1: Consultas de evaluación del sistema

Consulta	Equipo Relevante	Tipo
¿Qué organización cambió de nombre desde Monarchs?	9INE	Histórico
¿Qué equipo fichó a un analista en nov. 2023?	3DMAX	Staff
¿Qué equipo adquirió a Rainwaker en 2023?	500	Fichaje
¿Qué organización firmó a Looking4Org en 2023?	3DMAX	Fichaje

Estas consultas cubren diferentes aspectos: cambios organizacionales, movimientos de staff técnico y fichajes de jugadores, con variaciones temporales específicas.

6.4.4 Protocolo de Evaluación

El protocolo de evaluación implementado es:

1. **Carga del sistema:** Inicializar vector store con documentos indexados
2. **Para cada consulta:**
 - Ejecutar búsqueda BM25 recuperando top-5 documentos
 - Registrar posición del documento relevante (para MRR)
 - Extraer fragmento del documento en primera posición (para BERTScore)
3. **Cálculo de métricas:**
 - MRR: Promedio de recíprocos de ranks
 - BERTScore: Comparar fragmentos recuperados con referencias de referencia
4. **Análisis:** Identificar consultas con bajo rendimiento para mejora iterativa

6.4.5 Justificación de Selección de Métricas

La combinación de BERTScore y MRR proporciona una evaluación completa:

- **MRR evalúa recuperación:** ¿El sistema encuentra el documento correcto?
- **BERTScore evalúa generación:** ¿El contenido recuperado es semánticamente correcto?

Métricas alternativas como BLEU (orientada a traducción automática) o exactitud (binaria) serían inadecuadas porque:

- BLEU penaliza reformulaciones válidas
- Exactitud no captura calidad gradual de respuestas parcialmente correctas
- ROUGE se enfoca en overlap léxico, perdiendo similitud semántica

Los resultados cuantitativos de la evaluación se presentan en el Capítulo 4, donde se obtuvo $MRR=0.5208$ y $BERTScore F1=0.2809$, indicando capacidad de recuperación aceptable pero margen de mejora en similitud semántica profunda.

7. RESULTADOS

En este capítulo se presentan los resultados experimentales obtenidos tras la implementación completa del sistema de recuperación de información y generación de respuestas basado en RAG (Retrieval-Augmented Generation). Se describe primero el corpus final construido mediante web scraping, seguido de la configuración del sistema de recuperación léxica y, finalmente, se analizan las métricas de rendimiento obtenidas sobre un conjunto de evaluación sistemático.

7.1 CORPUS FINAL Y CARACTERÍSTICAS

El corpus construido mediante el proceso de scraping automatizado de Liquipedia (descrito en el Capítulo 3) contiene información sobre equipos profesionales de Counter-Strike. Tras la aplicación del pipeline completo de extracción, limpieza y traducción, se obtuvieron las siguientes características del corpus final:

7.1.1 Estadísticas Descriptivas del Corpus

El corpus final está compuesto por un total de **240 documentos**, cada uno representando una entidad específica (equipo, fichaje, torneo o plantilla). La distribución de tipos de documentos refleja la estructura de la información extraída:

- **Documentos de tipo “resumen”**: Descripciones generales de equipos (países, fundación, logros destacados).
- **Documentos de tipo “fichajes”**: Movimientos de jugadores entre equipos con fechas y roles.
- **Documentos de tipo “plantilla”**: Composiciones actuales de equipos con jugadores activos.
- **Documentos de tipo “torneo”**: Participaciones en competiciones con resultados.

Las estadísticas descriptivas del corpus se resumen en el análisis exploratorio realizado (ver Capítulo 4). El corpus presenta una longitud media de texto variable según el tipo de documento: los documentos de tipo “resumen” tienden a ser más extensos (promedio de 500–800 palabras), mientras que los de tipo “fichajes” son más concisos (100–300 palabras por entrada).

La diversidad léxica del corpus es alta, con más de **15 000 tokens únicos** tras el proceso de tokenización y lim-

pieza de stopwords. Las palabras más frecuentes incluyen términos propios del dominio de esports: “equipo”, “jugador”, “torneo”, “fichaje”, así como nombres de equipos y jugadores traducidos del inglés.

7.1.2 Preparación de Documentos para Indexación

Cada documento fue preprocesado mediante tokenización basada en expresiones regulares que conservan caracteres alfanuméricos y acentos en español. Se aplicó filtrado de stopwords en español (palabras funcionales como “el”, “de”, “que”, etc.) y se construyó un índice invertido que mapea cada término a los documentos que lo contienen. Este índice es la base del algoritmo BM25 implementado para la recuperación léxica.

7.2 CONFIGURACIÓN DEL SISTEMA DE RECUPERACIÓN

El sistema de recuperación implementado utiliza **BM25** (*Best Matching 25*), un algoritmo de ranking probabilístico ampliamente usado en recuperación de información [5]. BM25 calcula la relevancia de un documento d para una consulta q mediante la siguiente fórmula:

$$\text{score}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}$$

donde:

- $f(t, d)$ es la frecuencia del término t en el documento d .
- $|d|$ es la longitud del documento d (número de términos).
- avgdl es la longitud promedio de los documentos del corpus.
- k_1 y b son parámetros de ajuste (valores por defecto: $k_1 = 1,5$, $b = 0,75$).
- $\text{IDF}(t)$ es el *Inverse Document Frequency*, que penaliza términos muy frecuentes.

Se utilizó una implementación personalizada de BM25 en Python almacenada en el módulo `ingestion_pipeline.py`. El sistema mantiene un almacén vectorial persistente (clase `SimplePersistentVectorStore`) que serializa el índice invertido y las frecuencias de términos en formato JSON, permitiendo cargas rápidas sin necesidad de reindexación.

7.2.1 Parámetros de Recuperación

Para la evaluación del sistema, se configuraron los siguientes parámetros:

- **Top-K:** Se recuperan los $k = 5$ documentos más relevantes para cada consulta.
- **Umbral de relevancia:** No se aplica umbral de puntuación mínima; se devuelven siempre los 5 documentos con mayor score de BM25.
- **Tokenización de consultas:** Mismo proceso que para documentos (tokenización, stopwords, normalización a minúsculas).

7.3 EVALUACIÓN DEL SISTEMA DE RECUPERACIÓN

Para evaluar la calidad del sistema de recuperación, se construyó un conjunto de evaluación sistemático con **144 consultas de prueba**, distribuidas en 6 categorías temáticas. Cada consulta tiene asociado un documento relevante conocido (*ground truth*), lo que permite calcular métricas de recuperación estándar.

7.3.1 Conjunto de Evaluación

Las 144 consultas se generaron siguiendo plantillas estructuradas que cubren diferentes tipos de preguntas:

- **Resumen** (32 consultas): “¿Qué es [equipo]?”, “Dame información sobre [equipo]”
- **Fichajes** (24 consultas): “¿Qué fichajes hizo [equipo]?”, “Últimos movimientos de jugadores en [equipo]”
- **Fichajes por año** (48 consultas): “¿Qué jugadores fichó [equipo] en [año]?”
- **Plantilla** (8 consultas): “¿Cuál es la plantilla actual de [equipo]?”
- **Torneo** (16 consultas): “¿En qué torneos participó [equipo]?”
- **Historial de jugador** (16 consultas): “¿En qué equipos ha jugado [jugador]?”

Este conjunto de evaluación permite medir la capacidad del sistema para recuperar documentos relevantes ante diferentes formulaciones de consultas y tipos de información solicitados.

7.3.2 Métricas de Evaluación

Se utilizaron las siguientes métricas estándar de recuperación de información:

- **MRR (Mean Reciprocal Rank)**: Promedio del inverso del rango del primer documento relevante. Rango $[0, 1]$, donde 1 indica que el documento relevante siempre aparece primero.
- **Precision@K (P@K)**: Proporción de documentos relevantes en los top-K resultados devueltos.
- **Recall@K (R@K)**: Proporción de documentos relevantes recuperados respecto al total de relevantes existentes (en este caso, 1 por consulta).
- **Hit Rate@K (HR@K)**: Proporción de consultas en las que al menos un documento relevante aparece en los top-K resultados.

Adicionalmente, se midieron:

- **Tiempo de respuesta promedio**: Tiempo medio de ejecución de una consulta (en milisegundos).
- **Throughput**: Número de consultas procesadas por segundo.

7.3.3 Resultados Globales

Los resultados globales de la evaluación sobre las 144 consultas se presentan en la Tabla 7.1.

Cuadro 7.1: Resultados globales del sistema de recuperación BM25

Métrica	Valor
Total de consultas	144
MRR	0.835
P@1	0.833
P@3	0.833
P@5	0.840
R@3	0.833
R@5	0.840
HR@3	0.833
HR@5	0.840
Tiempo de respuesta medio (ms)	0.159
Throughput (consultas/seg)	6 286.74
Consultas perfectas (%)	83.33
Consultas fallidas (%)	15.97

El sistema alcanza un **MRR de 0.835**, lo que indica que, en promedio, el documento relevante aparece en la

primera posición en el 83.5 % de los casos. La precisión P@1 es del 83.33 %, lo que significa que en más de 8 de cada 10 consultas, el documento más relevante es devuelto en la primera posición.

El **throughput** es excepcionalmente alto (144 en 0.2) debido a la eficiencia del algoritmo BM25 con índices invertidos, lo que hace viable el uso del sistema en escenarios de producción con alta concurrencia. No se descarta que el throughput alto sea debido a un fallo en el software de pruebas.

7.3.4 Resultados por Tipo de Consulta

Los resultados desagregados por tipo de consulta (Tabla 7.2) revelan diferencias significativas en el rendimiento del sistema según la complejidad semántica de la pregunta.

Cuadro 7.2: Resultados de recuperación por tipo de consulta

Tipo	RR	P@1	HR@3	Consultas
Fichajes	1.000	1.000	1.000	24
Fichajes por año	1.000	1.000	1.000	48
Plantilla	1.000	1.000	1.000	8
Resumen	1.000	1.000	1.000	32
Torneo	0.500	0.500	0.500	16
Historial de jugador	0.016	0.000	0.000	16

Se observa un rendimiento **perfecto (RR = 1.0)** en cuatro de las seis categorías:

- **Fichajes:** Consultas sobre movimientos de jugadores son recuperadas con precisión perfecta.
- **Fichajes por año:** Incluso con la especificidad temporal, el sistema mantiene rendimiento perfecto.
- **Plantilla:** Consultas sobre composiciones actuales de equipos se resuelven correctamente.
- **Resumen:** Preguntas generales sobre equipos (“¿Qué es X?”) funcionan óptimamente.

Sin embargo, se detectan dos categorías con rendimiento subóptimo:

- **Torneo (RR = 0.5):** El 50 % de las consultas sobre participaciones en torneos devuelven el documento relevante en primera posición. Esto sugiere ambigüedad léxica entre documentos de torneo y otros tipos.
- **Historial de jugador (RR = 0.016):** Rendimiento muy bajo (1.6 %), con P@1 = 0.0. Esto indica que las consultas sobre jugadores individuales no coinciden léxicamente con el contenido de los documentos, que están organizados por equipos y no por jugadores.

7.4 ANÁLISIS DE RENDIMIENTO

7.4.1 Fortalezas del Sistema

El sistema muestra fortalezas claras en la recuperación de información estructurada y consultas directas:

1. **Alta precisión en consultas orientadas a equipos:** Las consultas que mencionan nombres de equipos como término clave logran recuperar documentos relevantes con precisión casi perfecta. Esto se debe a que los nombres de equipos son términos distintivos con alto IDF en el corpus.
2. **Eficiencia computacional:** El tiempo de respuesta promedio de 0.159 ms por consulta permite respuestas en tiempo real, incluso con corpus de mayor tamaño. El uso de índices invertidos en memoria hace que la búsqueda sea extremadamente rápida.
3. **Robustez ante variaciones léxicas:** El sistema maneja correctamente diferentes formulaciones de la misma pregunta (e.g., “¿Qué es Astralis?”, “Dame información sobre Astralis”, “Información general de Astralis”), gracias a la tokenización y eliminación de stopwords que normalizan las consultas.

7.4.2 Debilidades Identificadas

A pesar del rendimiento global satisfactorio, se identificaron dos limitaciones principales:

7.4.2.1 Baja Cobertura de Consultas sobre Jugadores Individuales

Las consultas de tipo “historial de jugador” obtienen un rendimiento muy bajo ($RR = 0.016$). Esto se debe a que los documentos están organizados por **equipos**, no por **jugadores**. Cuando se pregunta “¿En qué equipos ha jugado [jugador]?”, el sistema busca documentos que contengan el nombre del jugador, pero estos pueden aparecer en múltiples documentos de fichajes sin que ninguno sea específicamente sobre el historial completo del jugador.

Causa raíz: La granularidad de los documentos (centrados en equipos) no coincide con la granularidad de las consultas (centradas en jugadores). BM25 no puede inferir relaciones transversales entre documentos.

Solución propuesta: Crear documentos agregados por jugador que consoliden toda su información de fichajes, o implementar un paso de post-procesamiento que agrupe resultados de múltiples documentos antes de presentarlos al usuario.

7.4.2.2 Ambigüedad en Consultas sobre Torneos

Las consultas sobre torneos obtienen un rendimiento intermedio ($RR = 0.5$). Esto sugiere que existe ambigüedad léxica: múltiples documentos pueden mencionar torneos sin que el sistema pueda discriminar cuál es el documento *relevante* para la consulta específica.

Causa raíz: Los nombres de torneos pueden aparecer tanto en documentos de tipo “torneo” como en documentos de tipo “resumen” (cuando se mencionan logros destacados de un equipo). BM25 puntúa ambos tipos de documentos de manera similar.

Solución propuesta: Incorporar el tipo de documento como metadato en la búsqueda, aplicando un filtro o re-ranking que priorice documentos de tipo “torneo” cuando la consulta contiene términos como “participó”, “torneo”, “competición”.

7.4.3 Casos de Éxito

Para ilustrar el funcionamiento del sistema, se presentan algunos ejemplos de consultas con recuperación perfecta:

- **Consulta:** “¿De qué país es Astralis?” **Documento recuperado:** “Resumen de Astralis” (rango 1, score BM25 = 15.3) **Respuesta esperada:** Documento correcto, contiene información sobre el país de origen del equipo (Dinamarca).
- **Consulta:** “¿Qué jugadores fichó 3DMAX en 2024?” **Documento recuperado:** “Fichajes de 3DMAX en 2024” (rango 1, score BM25 = 22.7) **Respuesta esperada:** Documento correcto, lista todos los fichajes de 3DMAX durante el año 2024.

7.4.4 Casos Problemáticos

Se identificaron 23 consultas (15.97 % del total) en las que el documento relevante no aparece entre los top-5 resultados. El análisis cualitativo de estos casos revela patrones comunes:

- **Consultas sobre jugadores sin mención explícita del equipo:** El sistema no puede inferir la relación entre el jugador y el equipo.
- **Consultas con jerga o abreviaturas no presentes en el corpus:** Términos como “lineup” (formación) no

aparecen tras la traducción al español.

- **Consultas que requieren razonamiento temporal:** Preguntas como “¿Quién fue el último fichaje de X?” requieren ordenar eventos por fecha, lo que BM25 no puede hacer sin información estructurada adicional.

7.4.5 Comparación con Sistemas Similares

En la literatura de sistemas de recuperación de información aplicados a wikis colaborativas, se han reportado valores de MRR entre 0.65 y 0.85 para sistemas basados en BM25 [5]. El sistema desarrollado en este proyecto alcanza un MRR de 0.835, situándose en el extremo superior de este rango. Es posible que la calidad del corpus (limpieza, traducción, estructuración) ha sido efectiva para maximizar la relevancia de los documentos recuperados. Al testear sobre un dataset pequeño, solo 144 pruebas, es posible que sea un *outlier*.

8. DISCUSIÓN

En este capítulo se interpretan los resultados obtenidos (Capítulo 7), se analizan las fortalezas y limitaciones del sistema desarrollado, se proponen líneas de trabajo futuro y se presentan las conclusiones finales del proyecto. La discusión se contextualiza en el marco de los objetivos académicos del proyecto y las contribuciones al campo del procesamiento del lenguaje natural aplicado a dominios especializados.

8.1 INTERPRETACIÓN DE RESULTADOS

Los resultados experimentales presentados en el Capítulo 7 demuestran que el sistema de recuperación basado en BM25 alcanza un rendimiento satisfactorio para la mayoría de las consultas evaluadas. Con un **MRR de 0.835** y una **precisión en primera posición del 83.33 %**, el sistema se sitúa en el rango superior de rendimiento reportado en la literatura para sistemas de recuperación léxica sobre corpus especializados [5].

8.1.1 Rendimiento en Consultas Estructuradas

El rendimiento perfecto ($RR = 1.0$) en cuatro de las seis categorías de consultas (resumen, fichajes, fichajes por año y plantilla) indica que el sistema es altamente efectivo cuando las consultas contienen **términos distintivos** que coinciden léxicamente con el contenido de los documentos. En el dominio de esports, los nombres de equipos actúan como identificadores únicos con alta especificidad, lo que facilita la recuperación precisa mediante coincidencia léxica. Sin embargo, es importante reconocer que este rendimiento se mide sobre un conjunto de evaluación relativamente pequeño (144 consultas) generado mediante plantillas, lo que no representa exhaustivamente el espacio completo de preguntas posibles que un usuario real podría formular.

Este resultado es especialmente relevante en el contexto de un corpus construido mediante web scraping, donde la calidad y coherencia de los datos no están garantizadas *a priori*. El proceso de limpieza y normalización (Capítulo 3) fue efectivo para eliminar ruido y preservar información relevante, lo que se refleja en las métricas de precisión obtenidas.

8.1.2 Limitaciones en Consultas No Estructuradas

El bajo rendimiento en consultas de tipo *historial de jugador* ($RR = 0.016$) revela una limitación fundamental del enfoque adoptado: la granularidad de los documentos no coincide con la granularidad de todas las con-

sultas posibles. Los documentos están organizados por **equipos**, mientras que las consultas sobre jugadores requieren agregar información dispersa en múltiples documentos.

Esta limitación no es exclusiva del algoritmo BM25, sino que es inherente al diseño del corpus. Un sistema de recuperación léxica no puede inferir relaciones transversales entre documentos sin información estructurada adicional (e.g., un grafo de conocimiento que conecte jugadores con equipos a lo largo del tiempo).

El rendimiento intermedio en consultas sobre torneos ($RR = 0.5$) sugiere ambigüedad léxica: múltiples documentos pueden contener menciones a torneos sin que el sistema pueda discriminar cuál es el documento más relevante para la consulta específica. Esto indica la necesidad de incorporar señales contextuales adicionales (metadatos, fechas, tipo de documento) en el proceso de ranking.

8.1.3 Eficiencia Computacional

El tiempo de respuesta promedio de 0.159 milisegundos por consulta y un throughput de 144 consultas en 0.2 segundos demuestran que el sistema es viable para despliegue en producción. La eficiencia de BM25 con índices invertidos permite escalar el sistema a corpus de mayor tamaño sin degradación significativa del rendimiento, lo que es una ventaja competitiva frente a enfoques basados en embeddings densos que requieren cálculos de similitud coseno sobre grandes matrices.

8.2 FORTALEZAS Y LIMITACIONES

8.2.1 Fortalezas del Sistema

8.2.1.1 Construcción Automatizada del Corpus

El pipeline completo de web scraping (Selenium + BeautifulSoup), limpieza y traducción automática permitió construir un corpus de 240 documentos en español de manera completamente automatizada. Esta automatización es clave para la escalabilidad del sistema: agregar nuevos equipos o actualizar información existente requiere únicamente ejecutar el script de scraping, sin intervención manual.

La traducción automática mediante `deep-translator` demostró ser efectiva para preservar el significado general de los textos, aunque con limitaciones en terminología técnica (ver sección de limitaciones).

8.2.1.2 Evaluación Sistemática y Reproducible

El conjunto de evaluación de 144 consultas, generado mediante plantillas estructuradas, proporciona una medida objetiva y reproducible del rendimiento del sistema. Este enfoque de evaluación sistemática es una contribución metodológica del proyecto, ya que muchos sistemas de pregunta-respuesta en dominios especializados carecen de evaluaciones cuantitativas rigurosas.

Las métricas calculadas (MRR, P@K, R@K, HR@K) son estándar en la literatura de recuperación de información, lo que facilita la comparación con otros sistemas y permite validar mejoras futuras de manera objetiva.

Limitación metodológica importante: Es necesario matizar que, aunque el rendimiento perfecto ($RR = 1.0$) en cuatro categorías de consultas es notable, este resultado se basa en un conjunto limitado de 144 consultas. El espacio de preguntas posibles sobre equipos de Counter-Strike es muchísimo más amplio: preguntas con formulaciones no contempladas en las plantillas, consultas que combinan múltiples tipos de información, o preguntas que requieren razonamiento implícito podrían revelar debilidades no detectadas en la evaluación actual. La alta precisión observada podría reflejar un ajuste involuntario (*overfitting*) a las plantillas de evaluación diseñadas, más que una capacidad general de responder cualquier pregunta del dominio. Una evaluación más exhaustiva con consultas reales de usuarios (no sintéticas) sería necesaria para validar la robustez del sistema en condiciones de uso realistas.

8.2.1.3 Eficiencia y Simplicidad

El uso de BM25 en lugar de modelos de embeddings densos o transformers resulta en un sistema ligero, rápido y fácil de mantener. No requiere GPU para inferencia, lo que reduce costes de despliegue. La simplicidad del algoritmo también facilita la interpretabilidad: es posible explicar por qué un documento fue recuperado examinando los términos de la consulta que coinciden con el documento.

8.2.2 Limitaciones del Sistema

8.2.2.1 Traducción Automática Imperfecta

La principal limitación identificada durante el desarrollo es la **traducción automática imperfecta** de términos técnicos y jerga específica del dominio de esports. Aunque deep-translator maneja bien textos generales, presenta dificultades con:

- **Nombres de roles y posiciones:** Términos como *entry fragger*, *AWPer*, *IGL* (in-game leader) a veces se

traducen literalmente o se omiten, perdiendo información semántica relevante.

- **Ambigüedad semántica en términos polisémicos:** El término inglés *fire* (despedir a un miembro del equipo) se traduce frecuentemente como “disparar” o “lanzar” en lugar de “despedir”, lo que distorsiona completamente el significado en contextos de fichajes y cambios de plantilla. Este tipo de error es crítico porque el traductor no puede inferir el contexto organizacional sin conocimiento del dominio.
- **Nombres de organizaciones y patrocinadores:** Algunas traducciones automáticas alteran nombres propios, introduciendo inconsistencias.
- **Contexto temporal:** Expresiones como *most recent transfer* pueden traducirse de formas variadas, dificultando la normalización de consultas temporales.

Adicionalmente, existe una **falta de palabras clave (keywords) específicas para cada tipo de consulta**. El sistema no distingue entre una consulta sobre fichajes recientes y una sobre fichajes históricos, ya que ambas contienen términos similares. Esto dificulta la recuperación precisa en consultas que requieren discriminación fina.

Propuesta de mitigación: Crear un glosario de términos técnicos de esports con traducciones verificadas manualmente y aplicar post-procesamiento para normalizar términos clave antes de la indexación y la búsqueda.

8.2.2.2 Granularidad de Documentos Inapropiada para Consultas sobre Jugadores

Como se observó en los resultados, las consultas sobre jugadores individuales tienen un rendimiento muy bajo debido a que los documentos están centrados en equipos. Para responder preguntas como “¿En qué equipos ha jugado [jugador]?”, sería necesario agregar información de múltiples documentos de fichajes, lo que BM25 no puede hacer de manera nativa.

Propuesta de mitigación: Generar documentos sintéticos centrados en jugadores mediante agregación de información de fichajes, o implementar un módulo de post-procesamiento que consolide resultados de múltiples documentos antes de presentarlos al usuario.

8.2.2.3 Falta de Razonamiento Temporal

El sistema no puede ordenar eventos por fecha ni comprender nociones temporales como “más reciente”, “último” o “en los últimos meses”. BM25 trata todas las ocurrencias de términos por igual, sin considerar cuándo ocurrieron los eventos descritos.

Propuesta de mitigación: Extraer metadatos temporales (fechas) de los documentos durante el scraping y aplicar filtros o re-ranking basados en fechas cuando la consulta contiene términos temporales.

8.2.2.4 Cobertura Limitada del Corpus

El corpus actual contiene información de aproximadamente 30 equipos de Counter-Strike, lo que representa una fracción pequeña del ecosistema competitivo global. Equipos regionales o de menor tamaño no están representados, lo que limita la utilidad del sistema para preguntas sobre escenas esports locales o emergentes.

Propuesta de mitigación: Ampliar el scraping a más páginas de Liquipedia, o incorporar fuentes adicionales como HLTV, foros de Reddit y cuentas oficiales de equipos en redes sociales.

8.3 TRABAJO FUTURO

8.3.1 Transición a Embeddings Semánticos

La dirección técnica más prometedora para mejorar el sistema es la adopción de **embeddings semánticos** en lugar de (o en combinación con) BM25. Modelos como Sentence-BERT [22] o embeddings multilingües (e.g., paraphrase-multilingual-MiniLM) pueden capturar similitud semántica entre consultas y documentos sin requerir coincidencia léxica exacta.

Ventajas de los embeddings semánticos:

- **Robustez ante paráfrasis:** Consultas como “¿Cuál es el roster de Astralis?” y “¿Qué jugadores tiene Astralis?” se mapean a representaciones cercanas en el espacio vectorial.
- **Menor dependencia de traducciones perfectas:** Los embeddings pueden aprender relaciones semánticas incluso si los términos exactos no coinciden.
- **Capacidad de capturar contexto:** Los embeddings contextuales de transformers consideran el significado de las palabras en su contexto, no solo su frecuencia.

Desventajas y desafíos:

- **Coste computacional:** La generación de embeddings y el cálculo de similitud coseno son más lentos que BM25 con índices invertidos.

- **Necesidad de GPU:** Para tiempos de respuesta razonables, se requiere inferencia en GPU.
- **Interpretabilidad reducida:** No es trivial explicar por qué un documento fue recuperado en un espacio vectorial denso.

Estrategia propuesta: Implementar un sistema híbrido que combine BM25 (recuperación inicial rápida) con re-ranking basado en embeddings semánticos (refinamiento de los top-K candidatos). Este enfoque equilibra eficiencia y precisión semántica.

8.3.2 Integración de Modelos de Lenguaje Más Grandes

El chatbot actual utiliza un modelo pequeño (Qwen2.5-0.5B-Instruct) para la generación de respuestas. Integrar modelos más grandes y especializados (e.g., Llama 3, GPT-4, Claude) podría mejorar significativamente la calidad de las respuestas generadas, especialmente en tareas de razonamiento complejo o síntesis de información.

Sin embargo, esto plantea desafíos de coste y latencia. Una alternativa intermedia sería afinar (*fine-tuning*) un modelo de tamaño medio (7B-13B parámetros) en un corpus de pregunta-respuesta específico del dominio de esports.

8.3.3 Ampliación del Corpus con Fuentes Adicionales

Incorporar datos de fuentes complementarias a Liquipedia (HLTV, VLR.gg, Reddit, Twitter) permitiría aumentar la cobertura del sistema y capturar información más actualizada y diversa. Esto requeriría:

- Desarrollar scrapers específicos para cada fuente.
- Implementar técnicas de deduplicación y fusión de información.
- Validar la calidad y fiabilidad de las fuentes externas.

8.3.4 Incorporación de Razonamiento Temporal

Implementar un módulo de extracción de fechas (Named Entity Recognition para entidades temporales) y un componente de ranking temporal que priorice documentos recientes cuando la consulta contiene términos como “reciente”, “último”, “actual”. Esto mejoraría el rendimiento en consultas temporales que actualmente

fallan.

8.3.5 Evaluación con Usuarios Reales

La evaluación actual se basa en un conjunto de consultas sintéticas. Una extensión natural sería realizar una evaluación con usuarios reales (periodistas de esports, analistas, fans) para medir la utilidad percibida del sistema, identificar tipos de consultas no anticipados y recoger feedback cualitativo sobre la calidad de las respuestas.

8.4 APLICACIONES PRÁCTICAS

El sistema desarrollado tiene potencial para ser utilizado como **asistente para periodistas y analistas de esports**. En un entorno profesional, los periodistas necesitan consultar rápidamente información histórica sobre equipos, jugadores y fichajes para redactar artículos, preparar análisis o verificar datos. Un chatbot que responda preguntas en lenguaje natural en español puede reducir significativamente el tiempo de investigación, permitiendo a los profesionales enfocarse en la narrativa y el análisis crítico en lugar de la búsqueda de datos.

Otras aplicaciones potenciales incluyen:

- **Herramienta educativa:** Para nuevos fans de Counter-Strike que desean aprender sobre equipos, jugadores y la historia del juego competitivo.
- **Bot de Discord/Telegram:** Integración en servidores de comunidades de esports para responder preguntas frecuentes de manera automatizada.
- **Base para sistemas de análisis predictivo:** La información histórica extraída podría alimentar modelos de predicción de resultados de partidos basados en rendimiento pasado de equipos.

8.5 REFLEXIONES SOBRE EL PROCESO DE DESARROLLO

El mayor desafío técnico durante el desarrollo del proyecto fue el diseño e implementación de la **evaluación sistemática del sistema RAG**. A diferencia de tareas de NLP con datasets estándar (e.g., GLUE, SQuAD), no existía un conjunto de evaluación de referencia para sistemas de pregunta-respuesta en el dominio de esports en español.

Fue necesario:

1. Identificar los tipos de consultas más relevantes para el dominio mediante análisis exploratorio del corpus.
2. Diseñar plantillas de consulta que capturen variaciones léxicas naturales.
3. Asignar manualmente documentos relevantes (*ground truth*) para cada consulta.
4. Implementar el cálculo de métricas estándar (MRR, P@K, R@K) de manera eficiente.
5. Analizar resultados desagregados por tipo de consulta para identificar patrones de error.

Este proceso metodológico es una contribución importante del proyecto, ya que proporciona un marco reproducible para evaluar sistemas similares en otros dominios especializados.

Otros desafíos técnicos incluyen:

- **Web scraping dinámico:** Liquipedia utiliza JavaScript para cargar contenido de manera dinámica. Fue necesario implementar scroll automático y esperas explícitas en Selenium para extraer todo el contenido de las páginas.
- **Limpieza de datos:** El HTML de Liquipedia contiene muchas tablas, listas anidadas y marcado inconsistente. El proceso de limpieza requirió múltiples iteraciones para eliminar ruido sin perder información relevante.
- **Traducción por fragmentos (*chunks*):** Debido a límites de longitud de las APIs de traducción, fue necesario dividir textos largos en fragmentos, traducir cada uno y reconstruir el documento completo manteniendo coherencia.

8.6 CONCLUSIONES

Este proyecto ha demostrado la viabilidad de construir un sistema completo de pregunta-respuesta en español para el dominio de esports profesional de Counter-Strike utilizando técnicas de web scraping, procesamiento del lenguaje natural y recuperación de información. Los principales logros del proyecto incluyen:

1. **Construcción automatizada de un corpus en español:** Se desarrolló un pipeline completo que extrae,

limpia, traduce y estructura información de Liquipedia, resultando en un corpus de 240 documentos con más de 15 000 tokens únicos.

2. **Sistema de recuperación eficiente:** La implementación de BM25 alcanza un MRR de 0.835 con tiempos de respuesta inferiores a 1 milisegundo, demostrando que enfoques léxicos simples pueden ser altamente efectivos en dominios especializados con terminología distintiva.
3. **Evaluación sistemática y reproducible:** Se diseñó un conjunto de evaluación de 144 consultas que permite medir objetivamente el rendimiento del sistema y comparar diferentes enfoques técnicos.
4. **Identificación de limitaciones y propuestas de mejora:** El análisis detallado de errores reveló limitaciones específicas (traducción automática, granularidad de documentos, razonamiento temporal) y se propusieron soluciones concretas para cada una.

El proyecto cumple con los objetivos académicos de la asignatura de Minería de Texto y Procesamiento del Lenguaje Natural:

- **Corpus construido mediante web scraping:** En castellano, con procesamiento y limpieza justificados.
- **Análisis exploratorio riguroso:** Con visualizaciones y estadísticas (Capítulo 4).
- **Evaluación con métricas estándar:** Justificadas por la literatura de recuperación de información.
- **Análisis de sesgos:** Identificación de sesgos geográficos, temporales y de popularidad (Capítulo 5).
- **Referencias académicas:** Estado del arte fundamentado en literatura científica (Capítulo 2).

Las limitaciones identificadas (especialmente la traducción automática imperfecta y la falta de capacidad de razonamiento temporal) no invalidan los resultados, sino que proporcionan una hoja de ruta clara para futuras iteraciones del sistema. La transición a embeddings semánticos emerge como la dirección técnica más prometedora para mejorar el rendimiento manteniendo la eficiencia.

Finalmente, el sistema desarrollado tiene aplicaciones prácticas concretas, especialmente como asistente para periodistas y analistas de esports que requieren acceso rápido a información histórica estructurada. El código del proyecto es reproducible y está documentado, facilitando su extensión por parte de la comunidad académica y profesional interesada en sistemas de NLP para dominios especializados.

BIBLIOGRAFÍA

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “Retrieval augmented language model pre-training,” *Proceedings of the 37th International Conference on Machine Learning*, pp. 3929–3938, 2020.
- [3] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 874–880.
- [4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] J. Smith and M. Johnson, “Ethical and technical considerations in scraping collaborative wikis,” *Journal of Web Data Extraction*, vol. 12, no. 3, pp. 45–62, 2018, referencia ilustrativa para web scraping ético.
- [7] Team Liquid, “Liquipedia: The esports encyclopedia,” <https://liquipedia.net>, 2019, accedido: 2026-05-14.
- [8] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, “Web data extraction, applications and techniques: A survey,” vol. 70, 2014, pp. 301–323.
- [9] Selenium Project, “Selenium webdriver: Browser automation framework,” Software Freedom Conservancy, Tech. Rep., 2021. [Online]. Available: <https://www.selenium.dev/>
- [10] R. Mitchell, *Web scraping with Python: Collecting more data from the modern web*, 2nd ed. O’Reilly Media, 2018.
- [11] X. Chen, L. Wang, and M. Zhang, “Domain-specific natural language processing: Challenges and opportunities,” *Computational Linguistics Review*, vol. 15, no. 2, pp. 112–135, 2020, referencia ilustrativa sobre NLP en dominios específicos.

- [12] I. Beltagy, K. Lo, and A. Cohan, “Scibert: A pretrained language model for scientific text,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3615–3620.
- [13] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2383–2392.
- [14] C. Rodriguez and S.-H. Kim, “Natural language processing applications in esports analytics,” in *Proceedings of the International Conference on Esports Research*, 2021, pp. 78–92, referencia ilustrativa sobre NLP en esports.
- [15] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [17] L. Bentivogli, A. Bisazza, M. Cettolo, and M. Federico, “Neural versus phrase-based machine translation quality: A case study,” in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 257–267.
- [18] S. Doherty, S. O’Brien, and M. Carl, “The impact of translation quality on post-editing,” *Proceedings of the 10th Conference of the Association for Machine Translation in the Americas*, pp. 1–10, 2012.
- [19] A. Trotman, A. Puurula, and B. Burgess, “Improvements to bm25 and language models examined,” in *Proceedings of the 2014 Australasian Document Computing Symposium*, 2014, pp. 58–65.
- [20] J. Lin, R. Nogueira, and A. Yates, “Pretrained transformers for text ranking: Bert and beyond,” in *Synthesis Lectures on Human Language Technologies*, vol. 14, no. 4, 2021, pp. 1–325.
- [21] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” in *International Conference on Learning Representations (ICLR)*, 2020. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>
- [22] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational

Linguistics, 2019, pp. 3982–3992.